

Relation Before Realization

A Core Calculus of Persistent Typed Declarations for the Behavior of Physical Products

ZHU ZHEHAO

Formal development KCN-judu/BDL_FV (Lean 4.33.1, through Phase 22); paper revision of 2026-09-21

ABSTRACT

When the behavior of an interactive physical product is designed, the designer often knows *that* one product quantity determines another before knowing *how*: a lamp’s brightness follows its tilt, and other parts of the design can be built on that relationship long before its formula is chosen. This paper presents λ_{BDL} , the core calculus of the Behavior Design Language, in which a typed semantic relationship is the primary design object. A relationship is a persistent declaration — a stable identity, a signature between nominal *concepts*, a growable set of public commitments — that may exist and be depended upon while its realization is absent; a realization, when it arrives, *refines* the declaration rather than replacing it. The calculus gives this intermediate design state a precise semantics and proves that design progression respects it: refining a declaration preserves every client’s typing unconditionally and every client’s discharged commitment under a monotonicity condition on evidence that a mechanized counterexample shows necessary. Nominal concepts keep a relationship between *meanings* distinct from a relationship between the representations those meanings share, and a realization may construct only the concepts its own signature announces, so that realization cannot silently cross a semantic boundary the designer did not draw. Because relationships in a physical product hold over time, the calculus interprets declarations as streams in authored clock domains with one temporal primitive, and is deterministic and total on causal designs without exposing any scheduler order; because a computed value is not yet an effect, a relationship reaches the world only through an explicit drive edge with a single driver. Reusable behavior is a structured set of relationships instantiated by renaming, and composition adds no semantic machinery. All definitions, theorems and counterexamples are mechanized in Lean 4 with no `sorry` and no classical choice.

Keywords: design calculi, refinement, nominal types, reactive semantics, clock domains, logical relations, mechanized metatheory, Lean 4

1 INTRODUCTION

Consider a designer working out the behavior of a tilt-dimmed lamp. Early in the work they know five things that are not yet code, and each is a requirement the calculus must meet; we label them so that the rest of the paper can name the requirement each mechanism answers.

- **(D1) A relationship exists before its formula.** Brightness depends on tilt — a relationship whose formula is undecided and may stay undecided for weeks.
- **(D2) Others already depend on it, and the formula must not change what they relied on.** The light is driven by the relationship, a warming base reads it, a second lamp will reuse it; when the formula arrives it must not retroactively change what the earlier declaration meant to those parts.
- **(D3) Meanings are distinct even when representations coincide.** A tilt and a motor angle are both angles, and a brightness and an opacity are both numbers between zero and one, yet the relationship *tilt to brightness* is not the relationship *motor angle to brightness*, and no formula should be able to turn one into the other by accident.
- **(D4) The relationship holds over time, in an authored timing domain.** The tilt moves with the interaction; the value belongs to the design at the ticks of that domain and at no other.
- **(D5) The relationship eventually moves a physical light.** A computed value is not yet an effect; where the design meets the world must be a fact about the relationship, not a side effect of computing it.

These facts must attach to the relationship without collapsing the design into its implementation.

The first of these is the one that a programming-language presentation handles least naturally. In a functional language, writing

$$f : A \rightarrow B$$

is normally the first line of a definition; the signature anticipates, and is soon accompanied by, a computation $f\ x = \dots$. The signature has meaning on its own — it is checked, it documents, it constrains — but the object being authored is the function. In the design workflow above, the first line alone is already the artifact. `dimByTilt : Tilt -> Brightness` states that a relationship exists, what it connects, and which semantic boundary it draws; it says nothing about how the relationship is computed, and the designer has not yet decided. This is not an unfinished program. It is a complete design commitment at one level of detail, deliberately left open at the next.

This paper is about giving that state a semantics. We present λ_{BDL} , the core calculus of the Behavior Design Language (BDL), a language for designing the behavior of interactive physical products. Its organizing thesis is **relation before realization**: the primary authored object is a typed semantic relationship, represented in the calculus as a *persistent declaration* with a stable identity, a signature, and public commitments; a computation is one possible *realization* of that relationship, supplied later as a refinement. The calculus exists to make the intermediate state — declared, typed, depended upon, unrealized — typable, referenceable, composable, refinable, stable for its clients, and eventually realizable, without pretending that the computation already exists.

We are careful about what this claim is. Existing mechanisms provide every piece of the story: module signatures separate interface from implementation; abstract types hide representations; refinement and contract systems attach progressively stronger constraints; synchronous languages interpret definitions as clocked streams. The calculus does not show that any of these cannot express a declared-but-unrealized relationship. What it contributes is a direct, compositional semantics for a workflow in which *concept identity, interface commitment, delayed realization, temporal structure and physical effect coexist as facts about one object*, together with mechanized proofs that they interact as the workflow needs. A relationship is representationally a declaration in an environment, referred to by identity from terms; it is a first-class *design* object, not a first-class value that terms pass around, and the paper says so wherever the distinction matters.

1.1 One object, six constraints

A likely first reaction to the calculus is that it bundles unrelated mechanisms — a refinement order, nominal types, dimensions, clocks, output bindings, renaming. The answer that organizes this paper is that each is a constraint on the same object:

- **Refinement** governs how a relationship acquires commitments and a realization (§4). The refinement order is the mathematical form of design progression, and the stability theorems say that progress does not destroy the meaning of earlier decisions.
- **Nominal concepts and the construction grant** govern *what meanings* a relationship connects and *what* a realization is authorized to produce (§5). Representation is not meaning; a signature’s result concept is the only concept its realization may construct.
- **Dimensions** govern the arithmetic of representations once a concept is observed, and are orthogonal to concept identity (§5.3).
- **Clock domains** govern *when* a relationship’s value belongs to the design (§6). A relationship participates in an authored temporal structure, and the semantics must respect it without exposing a scheduler.
- **Outputs** govern when a relationship’s value becomes physical effect (§8). A computed value is not an effect; the drive edge is where the design meets the world, once per output.
- **Composition** governs how relationship structures are reused (§9). A behavior is a structured set of relationships; instantiation renames, binding connects, and flattening shows that nothing new is needed.

The same choice recurs on three of these axes: identity is nominal — of a concept, of a clock domain, of an output — and crossing an identity is always a visible artifact (a declared relationship, a transport with an initial value, a drive edge), never a coercion the implementation performs on the designer’s behalf.

1.2 Contributions

The contributions, in the order we develop them, each answer a design question with a formal mechanism and a theorem.

1. **Relationship-first declarations and refinement** (§2, §4). A design is an environment of declarations each of which may lack a realization; typing reads declarations through their signatures only. The refinement order (add a commitment, supply a realization, strengthen a realized interface with re-verification) formalizes progressive commitment. Theorem 3 shows that a client typed against a relationship before its realization is typed after it, with no hypothesis; Theorem 4 shows that a client’s discharged commitments survive refinement when the validation layer’s evidence is monotone in the environment; Theorem 5 is a mechanized counterexample showing the condition necessary.
2. **Semantic integrity of realizations** (§5). A relationship connects concepts, not representations. Nominal concept types with write-once representation bindings keep `Tilt -> Brightness` distinct from `MotorAngle -> Brightness`; the construction grant, derived from the declaration’s own signature, is the authority a realization has to produce a concept (Theorem 7); erasure to representations is sound, and no evaluation, memory or transport creates a concept tag (Theorem 12).
3. **Temporal interpretation of relationships** (§6). Declarations are interpreted as streams in nominal clock domains with one temporal primitive — read a domain at its last activation strictly before now. Evaluation is deterministic (Theorems 9, 14) and total on causal designs by a tick-indexed logical relation (Theorems 11, 14); the strictly-before rule keeps the scheduler unobservable, and its alternative is mechanically shown to expose it (Theorem 15).
4. **Derived rather than primitive design structure** (§7). Lists, products and one recursor suffice for the collection equations a designer writes, and the lossless cross-domain window — the construction most likely to be proposed as a primitive — is five declarations over memory, transport and lists, correct for every schedule (Theorem 16).
5. **Explicit physical effect** (§8). A relationship becomes effect only through a drive edge to a nominally identified output with a single driver; the physical output is then a function of the tick (Theorem 17), and hidden arbitration is shown to make three outputs from one design.
6. **Compositional reuse** (§9). Components, instances and bindings are derived from renaming and realization; the flattened composition is an ordinary design accepted by the unchanged judgments (Theorem 19), and modular and flat evaluation agree on a stated fragment (Theorem 20).
7. **Mechanization**. Every definition, theorem and counterexample is a Lean 4 declaration in `KCN-judu/BDL_FV` [1]; the development has no `sorry`, uses only propositional extensionality and quotient soundness (through function extensionality), and no classical choice. Each result is cited by its Lean name where it is stated; Appendix A indexes them.

Two conventions bound every claim. *Minimal* means minimal among the alternatives that were formalized and refuted, never a minimality theorem. Where a theorem holds for a fragment, the fragment is part of the statement. The paper is not about the surface language, the authoring environment, the toolchain, hardware validation or deployment, all of which live above the kernel and are described in the BDL monograph of which this paper is the formal core; and it makes no claim about designers — that a workflow is well served by this semantics is an empirical question no theorem here addresses.

2 THE DESIGN PROBLEM

2.1 A relationship as a design object

We fix vocabulary for the rest of the paper. A **relationship** is the design-level commitment: *brightness follows tilt*. A **declaration** is the formal object that represents it — a stable identity δ , an **interface** (the relationship’s public promise: an expected type between concepts and a list of commitments), and an optional **realization** (a computation implementing the promise). **Refinement** is the progressive strengthening of a declaration — more commitments, then a realization, then stronger commitments re-verified — under which clients that relied on the earlier state remain valid. A **design** is an environment of declarations. In the Lean development a declaration

without a realization is *unresolved*; the paper says *unrealized*, because the word matters: nothing is missing from such a declaration. Its identity, its concepts, its commitments and, later, its timing domain and output are all present. What is postponed is the computation, and postponing it is the designer’s decision.

Three kinds of declaration will appear. A relationship with inputs, $\text{dimByTilt} : \text{Tilt} \rightarrow \text{Brightness}$, has an arrow signature. A relationship with no inputs, $\text{light} : \text{Brightness}$, is a value the design computes. And a declaration that will *never* receive a realization, $\text{tilt} : \text{Tilt}$, is an *input*: a relationship the environment realizes, whose value the product reads. The kernel distinguishes none of these by kind; all are declarations, and the last two differ only in whether a realization is present. That an input is “a relationship realized by the world” is not a metaphor here — it is exactly how the semantics of §6 reads it.

In type-theoretic terms the whole design state is a *global environment* of named constants, and that is the vocabulary we use from here on. A **concept** is a *nominal base type*: a type constant C , distinct from every other by name, whose values are formed by the injection mk_C from a representation type R that the environment Θ binds to it — an abstract type with a private constructor (§5). A **declaration** is a *typed constant* $\delta : \tau$ in the global environment Δ , with an optional *definiens*: exactly a proof assistant’s *Parameter* before its *Definition*, except that here the parameter state is the normal one and giving the definiens is the design step. A declaration of base type, $\delta : C$, is one *inhabitant* of the concept — one value at each tick — and its definiens, when present, is the one term that produces that value; a declaration without a definiens is an *axiom* the environment discharges (the product’s *Source*). A declaration of function type is a template applied wherever another definiens names it. Several constants of one base type are ordinary ($\text{sensorA} : \text{Temperature}$, $\text{sensorB} : \text{Temperature}$, $\text{roomTemp} : \text{Temperature} := \text{if available then sensorA else sensorB}$); a term refers to a constant by name and never to a type, so nothing is ever resolved “by concept”. The product speaks the same ladder as *concept* (the type), *Sem block* (a constant of concept type, holding one value) and *mapping block* (its definiens); the calculus needs only *type*, *constant* and *definiens*, and it keeps the product’s word *relationship* for a constant read as a design object.

2.2 Unrealized but usable

The claim that an unrealized declaration is a complete design state is only worth making if the state is *usable*. In λ_{BDL} it is. Let the design Δ_0 contain

$$\begin{aligned} \text{dimByTilt} &= \langle \delta_1, \langle \text{Tilt} \rightarrow \text{Brightness}, [] \rangle, \text{none} \rangle, \\ \text{tilt} &= \langle \delta_2, \langle \text{Tilt}, [] \rangle, \text{none} \rangle, \end{aligned}$$

and let the designer now declare and *realize* the light:

$$\text{light} = \langle \delta_3, \langle \text{Brightness}, [] \rangle, \text{some} ((\delta_1) (\delta_2)) \rangle.$$

The realization of light is well typed in Δ_0 , by T-Ref twice — using only $\Delta_0 \ni \delta_1 : (\text{Tilt} \rightarrow \text{Brightness})$ and $\Delta_0 \ni \delta_2 : (\text{Tilt})$ — and T-App once:

$$\begin{aligned} &(\text{T-Ref}) \Theta; \Delta_0; [] \vdash_G \delta_1 : \text{Tilt} \rightarrow \text{Brightness} \\ &(\text{T-Ref}) \Theta; \Delta_0; [] \vdash_G \delta_2 : \text{Tilt} \\ &(\text{T-App}) \Theta; \Delta_0; [] \vdash_G (\delta_1) (\delta_2) : \text{Brightness} \end{aligned}$$

The derivation consults Δ_0 only through the *type view* $\Delta_0 \ni \delta : \tau$ — the expected type of each declared identity — and never asks whether δ_1 has a realization. light is a well-formed, typed, referenceable part of the design while dimByTilt has no formula. This is the calculus’s thesis made formal, and the theorem that completes it is stated in §4: when dimByTilt is later realized, or given a commitment, every judgment about light in Δ_0 holds in the refined design (Theorem 3). The client did not depend on a body, so no body can invalidate it.

2.3 Design progression as refinement

The running example progresses through eight commitments, in the order a designer might make them; each is a step in the calculus and each is taken up in a later section.

step	the designer commits to	the calculus records	§
1	a relationship $\text{dimByTilt} : \text{Tilt} \rightarrow \text{Brightness}$ exists	an unrealized declaration with that signature	2.2, 4

step	the designer commits to	the calculus records	§
2	light is what <code>dimByTilt</code> yields for the current tilt	a realized declaration referring to <code>dimByTilt</code> by identity	2.2
3	Tilt is represented by an angle, Brightness by a scalar	write-once bindings in the concept environment Θ	5
4	a formula for <code>dimByTilt</code>	a realization, typed under the grant of its own signature	4, 5
5	<code>dimByTilt</code> promises monotone	a commitment, with evidence re-verified	4
6	light updates with the interaction, in domain <code>fast</code>	a clock assignment and the domain judgment	6
7	the lamp's LED shows light	a drive edge to an output with a single driver	8
8	a second lamp reuses the whole behavior	a component instantiated by renaming and bound	9

Table 1. The running example as progressive commitment.

Conceptually, a design before all its realizations are chosen describes a constrained family of completed behaviors, and each step narrows the family: a commitment excludes realizations that lack the property, a realization fixes one computation, a clock assignment fixes when values are observed, a drive edge fixes what the product does. We use this reading as motivation only. The calculus does not denote a set of possible products; what it has is a *refinement relation* on declarations and environments (§4.2), and the theorems are about that relation — one direction of progressive commitment, from less determined to more. Steps that do not narrow — changing a signature, dropping a commitment, replacing a realization — are *edits*, and the calculus promises nothing about them (§4.4).

3 THE CALCULUS

The term language of λ_{BDL} is a simply typed λ -calculus with registered first-order operators. What makes it a calculus of relationships rather than of functions is not its terms but the environments they are typed and evaluated against, and the projections through which each judgment may read them. This section fixes the syntax, the environments and the typing judgment; §3.3 then lists the known type-system features the judgment assembles, each with its source, the counterexample that makes it necessary and the theorem that makes it sufficient. The design content of each environment is developed in the sections that follow.

3.1 Syntax

Before the grammar, the shape of a design in words. A design is a table of named constants; each row carries a name, a type, a list of commitment labels, and either a term or nothing. A concept is a type name paired, once, with the representation type its values are stored in. Time is one global counter of ticks with a schedule saying which named domains activate at which ticks; each constant belongs to one domain or to none. An output is a name that accepts one type in one domain and is driven by at most one constant. The running example is three rows — `dimByTilt`, `tilt`, `light` — one concept binding for each of `Tilt` and `Brightness`, one domain `fast`, and one output `led`. The grammar and the judgments that follow make each of these precise, and nothing else is in the calculus.

Figure 1 gives the syntax. Types are those of a simply typed calculus with booleans, counts and arrows, extended by nominal base types — the concepts C , type constants distinct by name — physical quantities Q_d over a dimension d , and the data formers `Option`, `List` and $\times$. A dimension is an exponent vector over a fixed finite set of base dimensions (length, time, angle, mass, temperature in the development); dimensions form an abelian group under pointwise addition, which is the only structure the calculus uses.

$$\begin{aligned}
& C \in \text{ConceptId} \quad d \in \text{Dim} \quad \kappa \in \text{ClockId} \quad \delta \in \text{DeclId} \quad o \in \text{OutputId} \\
& \tau, \sigma ::= \text{Bool} \mid \text{Nat} \mid \tau \rightarrow \sigma \mid C \mid Q_d \mid \text{Option } \tau \mid \text{List } \tau \mid \tau \times \sigma \\
& e ::= x \mid \text{true} \mid \text{false} \mid n \mid \lambda x : \tau. e \mid e e \mid \delta \mid \text{rep } e \mid \text{mk}_C e \mid p \\
& \quad \mid \text{delay } e e \mid \text{sync}_\kappa e e \mid \text{fold } e e e \\
& p ::= \text{lit}_d n \mid \text{add}_d \mid \text{sub}_d \mid \text{mul}_{d_1 d_2} \mid \text{div}_{d_1 d_2} \mid \text{lt}_d \mid \text{eq}_\tau^{pf} \mid \neg \mid \wedge \mid \vee \mid \text{ite}_\tau \\
& \quad \mid \text{none}_\tau \mid \text{some}_\tau \mid \text{isSome}_\tau \mid \text{getD}_\tau \mid \text{nil}_\tau \mid \text{cons}_\tau \mid \text{length}_\tau \mid \text{take}_\tau \mid \text{drop}_\tau \mid \text{reverse}_\tau \mid \text{head}_\tau \\
& \quad \mid \text{toList}_\tau \mid \text{pair}_{\tau\sigma} \mid \text{fst}_{\tau\sigma} \mid \text{snd}_{\tau\sigma}
\end{aligned}$$

Figure 1. Syntax of λ_{BDL} . Variables are de Bruijn indices in the development; we write names. The superscript pf on eq_τ^{pf} is a proof that τ is a data type.

Notation. The conventions are those of type theory, and each letter is bound once, where its object first appears, and never rebound. Metavariables: C a concept — a nominal base type, the level of ConceptId ; δ a constant name (a declaration identity) and h a declaration record (§3.2); τ, σ types and R a representation, a concept-free data type; e, b, i terms, x variables, v, w values — a value of concept C is $\text{mk}_C v$ (§6); κ a clock domain (lowercase c is avoided, so that no letter reads as an inhabitant of the concept C), o an output, d a dimension, t a tick, p a property, n a numeral. Environments: Θ the signature of concepts ($\Theta(C) = R$ binds a representation), Δ the global environment of constants ($\Delta \ni \delta : \tau$ declared, $\Delta \ni \delta := b$ defined), Γ the local context, G the grant (a set of concepts, written on the turnstile: $\Theta; \Delta; \Gamma \vdash_G e : \tau$), K clocks, S the schedule, I the input, ρ the evaluation environment, Ω outputs and β drive edges (§3.2, §8); $\mathcal{P} = \langle \tau, \mathcal{K} \rangle$ an interface with its commitment list (§4), ev evidence, η an erasure (§5.2), $\mathcal{C}$ a component and k an instance index (§9). Keywords and named judgments are set in sans serif (mk , sync_κ , GlobalWF). The ladder of §2 reads, in these letters: R is the type of a type, C is a type, δ is one inhabitant holding one v per t .

Terms are those of the λ -calculus plus five design-specific forms and one recursor. A constant name δ refers to a relationship by the identity of its declaration; nothing about the declaration’s interface or realization is in the syntax, which is what lets a term refer to a relationship that has no realization yet. The term $\text{rep } e$ observes the representation of a concept value and $\text{mk}_C e$ constructs one — the elimination and the introduction of the abstract type C ; the value form is $\text{mk}_C v$. The term $\text{delay } i e$ is the value of e at the previous activation of the current domain, i before any; $\text{sync}_\kappa i e$ is the value of e in domain κ at κ ’s last activation strictly before now, i if none. The term $\text{fold } f z l$ is the list recursor: $\text{fold } f z [x_1, \dots, x_n] = f x_1 (\dots (f x_n z))$. The recursor is primitive because it cannot be derived: the term language is the simply typed λ -calculus, which has no fixpoint combinator and no general recursion — a definiens may not refer to itself instantaneously (§6.2, Proposition 10), and the calculus is meant to be total (Theorem 11) — so abstraction and application alone define no closed term that iterates over a list of unknown length; an inductive type needs its eliminator, and fold is the one eliminator, from which every collection operation is defined (§7.1). It is a term former rather than a registered operator because it applies a function value in the course of evaluation, which the first-order operators p never do. Registered operators p are first-order constants with types; they never apply a closure.

Two predicates on types recur. A type is **data**, τ . **Data**, when no arrow occurs in it; a type is **concept-free**, τ . **SemFree**, when no concept C occurs in it. Both are decidable by structural recursion, and $(\tau \times \sigma)$. **Data** $\Leftrightarrow \tau$. **Data** $\wedge \sigma$. **Data**, (**List** τ). **Data** $\Leftrightarrow \tau$. **Data** hold definitionally (Ty.prod_data , Ty.list_data).

3.2 Environments

A term is typed and evaluated against several environments, each read through a stated projection and nothing else. This discipline — *which environment a judgment may see* — is what the stability results of §4 rest on: a client sees a relationship’s signature and never its realization, so the realization can change without the client noticing.

- A **declaration** is a triple $h = \langle \delta, \mathcal{P}, r \rangle$: a constant name δ , an interface $\mathcal{P} = \langle \tau, \mathcal{K} \rangle$ — the expected type and a list of commitments, atomic property labels — and an optional definiens r , which is none or some b for a term b . A **design** is a global environment Δ of declarations, a partial map from constant names. Two projections of it are the only views a judgment may take: the *type view* $\Delta \ni \delta : \tau$ (the constant δ is declared at τ), which is all that typing sees, and the *realization view* $\Delta \ni \delta := b$ (its definiens is b), which is all that

evaluation sees; $\delta \notin \text{def}(\Delta)$ says the constant has no definiens. An **unrealized** declaration is one without a definiens; nothing else distinguishes it.

- A **concept signature** Θ binds each concept, write-once, to a representation, $\Theta(C) = R$. It is well formed, Θ . WF, when every bound representation is concept-free and data: $\Theta(C) = R \Rightarrow R$. SemFree $\wedge$ R. Data.
- A **grant** G is a set of concepts a term may construct. The empty grant $\emptyset$ permits nothing; $\text{grant}(\tau)$ permits the concepts in result position of τ : $\text{grant}(C) = \{C\}$, $\text{grant}(\tau \rightarrow \sigma) = \text{grant}(\sigma)$, and $\text{grant}(\tau) = \emptyset$ otherwise.
- A **clock environment** $K : \text{DeclId} \rightarrow \text{Option ClockId}$ assigns each declaration a domain; none marks a domain-agnostic relationship usable in any domain. A **schedule** $S : \text{ClockId} \rightarrow \mathbb{N} \rightarrow \text{Bool}$ says at which global ticks each domain activates. An **input** $I : \text{DeclId} \rightarrow \mathbb{N} \rightarrow \text{Value}$ supplies a value for every unrealized declaration at every tick — the environment's realization of the design's inputs.
- An **output environment** $\Omega : \text{OutputId} \rightarrow \text{Option } \langle \text{accepts} : \text{Ty}, \text{clock} : \text{ClockId} \rangle$ and the **drive edges** $\beta : \text{DeclId} \rightarrow \text{Option OutputId}$ are introduced in §8.

Typing sees Θ , the type view of Δ and G . Evaluation sees the realization view of Δ , I and (in several domains) S . The domain judgment sees K . Outputs see Ω , K , the type view of Δ and β . Commitments and evidence are seen by the satisfaction relation of §4 and by nothing else.

3.3 Typing

The typing judgment $\Theta; \Delta; \Gamma \vdash_G e : \tau$ is given in Figure 2. Rules T-Var, T-Bool, T-Nat, T-Lam and T-App are those of the simply typed λ -calculus. T-Ref is the only rule that reads Δ , and it reads the type view. T-Rep and T-Mk read Θ through the binding $\Theta(C) = R$; T-Mk additionally requires the grant. T-Prim assigns each registered operator its type; the dimension algebra is entirely in that table (Figure 3), so an application of $\text{mul}_{d_1 d_2}$ is checked by T-App like any other. T-Delay and T-Sync require the type to be data and the context to be empty. T-Fold types the recursor, the only rule beyond T-App under which a function value is applied; since the language has no fixpoint, it is also the only source of iteration (§3.1).

$$\begin{array}{c}
\frac{\Gamma(x) = \tau}{\Theta; \Delta; \Gamma \vdash_G x : \tau} \text{ (T-Var)} \quad \frac{}{\Theta; \Delta; \Gamma \vdash_G b : \text{Bool}} \text{ (T-Bool)} \quad \frac{}{\Theta; \Delta; \Gamma \vdash_G n : \text{Nat}} \text{ (T-Nat)} \\
\frac{\Theta; \Delta; \Gamma, x : \tau \vdash_G e : \sigma}{\Theta; \Delta; \Gamma \vdash_G \lambda x : \tau. e : \tau \rightarrow \sigma} \text{ (T-Lam)} \quad \frac{\Theta; \Delta; \Gamma \vdash_G f : \tau \rightarrow \sigma \quad \Theta; \Delta; \Gamma \vdash_G a : \tau}{\Theta; \Delta; \Gamma \vdash_G f a : \sigma} \text{ (T-App)} \\
\frac{\Delta \ni \delta : \tau}{\Theta; \Delta; \Gamma \vdash_G \delta : \tau} \text{ (T-Ref)} \quad \frac{}{\Theta; \Delta; \Gamma \vdash_G p : \text{ty}(p)} \text{ (T-Prim)} \\
\frac{\Theta(C) = R \quad \Theta; \Delta; \Gamma \vdash_G e : C}{\Theta; \Delta; \Gamma \vdash_G \text{rep } e : R} \text{ (T-Rep)} \quad \frac{C \in G \quad \Theta(C) = R \quad \Theta; \Delta; \Gamma \vdash_G e : R}{\Theta; \Delta; \Gamma \vdash_G \text{mk}_C e : C} \text{ (T-Mk)} \\
\frac{\tau. \text{Data} \quad \Theta; \Delta; [] \vdash_G i : \tau \quad \Theta; \Delta; [] \vdash_G e : \tau}{\Theta; \Delta; [] \vdash_G \text{delay } i e : \tau} \text{ (T-Delay)} \\
\frac{\tau. \text{Data} \quad \Theta; \Delta; [] \vdash_G i : \tau \quad \Theta; \Delta; [] \vdash_G e : \tau}{\Theta; \Delta; [] \vdash_G \text{sync}_\kappa i e : \tau} \text{ (T-Sync)} \\
\frac{\Theta; \Delta; \Gamma \vdash_G f : \tau \rightarrow \sigma \rightarrow \sigma \quad \Theta; \Delta; \Gamma \vdash_G z : \sigma \quad \Theta; \Delta; \Gamma \vdash_G l : \text{List } \tau}{\Theta; \Delta; \Gamma \vdash_G \text{fold } f z l : \sigma} \text{ (T-Fold)}
\end{array}$$

Figure 2. Typing (HasType). T-Ref is the only rule reading Δ ; T-Rep and T-Mk the only rules reading Θ ; T-Mk the only rule reading G .

operator	type	operator	type
$\text{lit}_d n$	Q_d	eq_τ^{pf}	$\tau \rightarrow \tau \rightarrow \text{Bool}$
$\text{add}_d, \text{sub}_d$	$Q_d \rightarrow Q_d \rightarrow Q_d$	ite_τ	$\text{Bool} \rightarrow \tau \rightarrow \tau \rightarrow \tau$
$\text{mul}_{d_1 d_2}$	$Q_{d_1} \rightarrow Q_{d_2} \rightarrow Q_{d_1+d_2}$	some_τ	$\tau \rightarrow \text{Option } \tau$
$\text{div}_{d_1 d_2}$	$Q_{d_1} \rightarrow Q_{d_2} \rightarrow Q_{d_1-d_2}$	getD_τ	$\text{Option } \tau \rightarrow \tau \rightarrow \tau$
lt_d	$Q_d \rightarrow Q_d \rightarrow \text{Bool}$	toList_τ	$\text{Option } \tau \rightarrow \text{List } \tau$

operator	type	operator	type
length_τ	$\text{List } \tau \rightarrow Q_0$	cons_τ	$\tau \rightarrow \text{List } \tau \rightarrow \text{List } \tau$
head_τ	$\text{List } \tau \rightarrow \text{Option } \tau$	$\text{take}_\tau, \text{drop}_\tau$	$Q_0 \rightarrow \text{List } \tau \rightarrow \text{List } \tau$
$\text{fst}_{\tau\sigma}$	$\tau \times \sigma \rightarrow \tau$	$\text{pair}_{\tau\sigma}$	$\tau \rightarrow \sigma \rightarrow \tau \times \sigma$

Figure 3. Types of the registered operators (Prim.ty), abridged. Dimension algebra lives here and nowhere else. Equality eq is available at every data type and the order lt at quantities only.

3.3.1 The type system as an assembly: each part, its source, its necessity and its sufficiency

Nothing in Figure 2 is new as a mechanism. The type system is an assembly of known features, each taken from a system where it is standard, cut down to the smallest form that meets one of the requirements D1–D5 of §1, and combined with the others; the calculus is the combination, and the theorems are about the combination. A reader who expects the simply typed λ -calculus is owed, for each part, three things: where it comes from, why it is *necessary* – what goes wrong without it, as a mechanized counterexample – and why it is *sufficient* – the theorem that then holds with no further rule. The parts are listed in the order the paper develops them; the sections named carry the details.

1. **Typed constants with an optional definiens, typed by the declaration alone** (T-Ref reads the type view and never a definiens). *Source*: the Parameter/Definition distinction of proof assistants, and the signature/structure separation of ML modules [2], [3], without the module language – the environment is flat and identity is the name. *Requirement*: D1. **Necessary**: if the rule could see a definiens, a client’s derivation would depend on it, and supplying or changing a definiens could invalidate the client – the state the calculus exists for is the one in which the definiens is absent. **Sufficient**: with this rule alone every client’s typing survives every refinement of the declaration it reads (Theorem 3, no side condition); the boundary is exact, since retyping a declaration breaks clients, which is why the type is frozen in the refinement order and retyping is an edit (§4.4).
2. **Interfaces ordered by refinement – type frozen, commitments growing – with satisfaction by an abstract evidence relation** ($\mathcal{P} \sqsubseteq \mathcal{P}'$, Satisfies; §4). *Source*: contract and refinement systems for the growing half, manifest types [3] for the frozen half, without refinement types – commitments are atomic labels, and what discharges them is the validation layer’s, not the kernel’s. *Requirement*: D2. **Necessary**: strengthening a realized interface without re-verifying the definiens breaks well-formedness (naive_breaks_wellformedness), and evidence that consults the *absence* of a definiens is destroyed by a legal step (Theorem 5) – the order and the monotonicity condition are both load-bearing. **Sufficient**: the lifecycle closure is exactly the structural order (Proposition 2), and under monotone evidence a globally well-formed design stays so through every step and every lifecycle (Theorem 4).
3. **Nominal base types with a write-once representation** (two concepts of one representation are distinct types; T-App rejects one where the other is expected; §5.1). *Source*: abstract types with a private constructor [4], [5], the constructor exported not to a module but to one declaration. *Requirement*: D3. **Necessary**: with concepts identified by representation, the wire `motorTarget := tiltSensor` between two angle-valued concepts is well typed and the design globally well formed (counterexampleA_baseline_accepts_invalid_wire); the model then records nothing of the distinction the designer drew. **Sufficient**: T-App alone rejects the wire (semantic_identity_mismatch_rejected), and the explicit relationship `tiltToMotor : Tilt -> MotorAngle` is an ordinary declaration accepted by the same rules (explicit_semantic_mapping_accepted); no compatibility judgment is added.
4. **Construction granted, observation free** (T-Mk requires $C \in G$, T-Rep does not; a definiens of type τ is typed under $\text{grant}(\tau)$, client code under $\emptyset$; §5.1). *Source*: capability-style typing over the private constructor of item 3, with capabilities that are a set of type names on the turnstile – never values, never passed, never abstracted over – computed from the declaration’s own signature, so that no annotation is written. *Requirement*: D3, for the realization. **Necessary**: with mk_C available everywhere, $\lambda x. \text{mk}_{\text{Motor}}(\text{rep } x)$ is a well-typed `Tilt -> MotorAngle` in the empty environment, and the crossing can hide inside a body whose signature names no motor (unrestricted_representation_binding_bypasses_semantic_identity, hidden_crossing_inside_unrelated_body); observation alone is safe but can realize no relationship. **Sufficient**: a well-typed term constructs only the concepts its grant lists (Theorem 7), so a definiens

constructs only the concept its signature announces; the hidden crossing is rejected and becomes legal exactly when `tiltToMotor` is declared (`hidden_crossing_rejected_under_grant`).

5. **Dimensions as type indices, the algebra in the operator types, no dimension rule** (Q_d over an abelian group; $\text{mul}_{d_1 d_2} : Q_{d_1} \rightarrow Q_{d_2} \rightarrow Q_{d_1+d_2}$; T-App checks the application; §5.3). *Source*: units of measure [6], [7], without dimension polymorphism or inference — the kernel has no unit variables, and units are a surface elaboration. *Requirement*: D3, for the arithmetic under a concept. **Necessary**: without dimensions in types, adding a length to a time is well typed (`counterexampleB_baseline_accepts_length_plus_time`). **Sufficient**: T-App rejects it (`dimension_mismatch_rejected`), a velocity is typed as a quotient (`velocity_typed`), dimensions stay orthogonal to concept identity (`same_dimension_does_not_imply_same_semantic_identity`), and erasing them preserves typing (`HasType.eraseDim`).
6. **Declarations as clocked streams, one temporal primitive, a domain judgment beside typing** (`delay`, `syncκ`, `Clocked`; §6). *Source*: synchronous dataflow — `pre` and clocks in Lustre [8], [9], determinism in the sense of Kahn [10] — without a clock calculus of sampling operators and without rates: one global tick and a schedule. *Requirement*: D4. **Necessary**: a transport that may see a simultaneously active source makes the scheduler’s order observable (Theorem 15), and memory without an explicit initial value leaves the first tick undefined or nondeterministic (`first_tick_undefined_without_init`, `first_tick_nondeterministic_without_init`). **Sufficient**: with strictly-before and explicit initial values, evaluation is deterministic and total in every domain for every schedule (Theorem 14), and `delay` is `sync` at the own domain, so one primitive suffices (Proposition 13).
7. **Memory and transport typed only at data types and only in the empty context** (T-Delay, T-Sync). *Source*: the placement of `pre` in Lustre, in nodes rather than in functions [8]. *Requirement*: D4, with totality. **Necessary**: a delayed value of arrow type would be a closure transported across ticks, and the logical relation at arrow type is indexed by the tick and cannot be transported (`arrow_not_delayable`); a delay under a binder would evaluate its operand at the previous tick in an environment created at the current one, which no rule can value (`delay_not_under_binder`, `sync_not_under_binder`). **Sufficient**: with the two restrictions evaluation is total on causal designs (Theorem 11), the relation at data types is independent of the tick (`Red_data`), and a delayed value carries the tag of the value delayed (Theorem 12).
8. **No fixpoint; list elimination is a primitive recursor** (T-Fold; no rule for recursion; §7.1). *Source*: the recursor of Gödel’s System T, and the causality analyses of synchronous languages for the rank of §6.2. *Requirement*: D4 — a stream must have a value at every tick. **Necessary**: the simply typed fragment has no self-application, and a self-referential definiens is either rejected by causality or has no value (Proposition 10), so abstraction and application define no closed term that iterates over a list of unknown length; and a recursor cannot be a registered operator, since operators are first-order and never apply a closure. **Sufficient**: one recursor is total on related values (`fold_total`) and derives every collection operation with its specification (`fold_spec`), while recursion *in time* remains available through memory (§6.2).
9. **Outputs as a separate sort with a per-declaration drive edge and a global single-driver invariant** (§8). *Source*: the single-assignment discipline of hardware description and dataflow, without effect types, action values or arbitration policies. *Requirement*: D5. **Necessary**: with several drivers, first-wins, last-wins and maximum give three physical outputs from one design (`hidden_arbitration_observable`). **Sufficient**: under the invariant the physical output is a function of the tick (Theorem 17), and binding an undriven output is a refinement (`first_output_binding_is_monotone`).
10. **Instantiation as renaming of every owned identity, binding as realization** (§9). *Source*: fresh-name instantiation and the flattening of hierarchical dataflow, with every judgment equivariant. *Requirement*: D2, for reuse. **Necessary**: with the display name as the identity, renaming breaks clients (`rename_under_name_identity_breaks_client`); with anything less than a renaming of every owned identity, two instances collide. **Sufficient**: every judgment is preserved by renaming (Proposition 18), and a flattened system is an ordinary design accepted by the unchanged judgments (Theorem 19).

Three features a reader might expect are absent on purpose. There is no polymorphism in the kernel: generic definitions are families instantiated by *matching* against the unique inferred type, which Proposition 1 makes possible (§7.2). There is no subtyping: the refinement order freezes the type, and every crossing between concepts, domains or outputs is a declared artifact, never a coercion. And there is no effect system: the only effect is the

drive edge, which is not a term. Each absence follows the paper’s method — a feature is added only when a formalized alternative without it was refuted — and everything else in Figure 2 is the standard system: T-Var, T-Bool, T-Nat, T-Lam and T-App are unchanged, and inference is syntax-directed (§3.4).

3.4 Inference, uniqueness and monotonicity

Inference is syntax-directed. A function $\text{infer } \Theta \Delta G \Gamma e : \text{Option Ty}$ follows the rules of Figure 2 and needs only decidability of G , of type equality and of τ . **Data.**

PROPOSITION 1 (Inference; `infer_sound`, `infer_complete`, `HasType.unique`). $\text{infer } \Theta \Delta G \Gamma e = \text{some } \tau$ iff $\Theta; \Delta; \Gamma \vdash_G e : \tau$; hence typing is decidable and every term has at most one type.

Proof. Define `infer` by structural recursion on e , reading each rule of Figure 2 as an equation: a variable looks up Γ ; a constant δ looks up $\Delta \ni \delta : \tau$; an application infers f and a and requires the inferred type of f to be an arrow $\tau \rightarrow \sigma$ with τ the inferred type of a ; `rep` e requires the inferred type to be a concept C and returns $\Theta(C)$; `mkC` e requires $C \in G$ and the inferred type to be $\Theta(C)$; a primitive returns its registered type. *Soundness* ($\text{infer} = \text{some } \tau$ implies a derivation) is by induction on e : in each case the equation’s premises are exactly the rule’s premises, and the induction hypotheses supply the sub-derivations. *Completeness* (a derivation implies `infer` returns its type) is by induction on the derivation: each rule’s premises determine the recursive calls, and the side conditions the rule checked are the ones `infer` tests. Uniqueness follows from completeness: two derivations of $e : \tau_1$ and $e : \tau_2$ give $\text{infer } e = \text{some } \tau_1 = \text{some } \tau_2$. Decidability follows from soundness and completeness together, since `infer` is a total computable function whose result is compared with τ . $\square$

Uniqueness matters beyond decidability: it is why the surface language’s polymorphism can be *matching* rather than unification (§7.2), and why a nominal mismatch is reported as “Brightness and Opacity are different concepts” and never as a unification residue. Typing is moreover monotone in each of its three environments — under environment refinement (§4.2), under binding more concepts, and under a larger grant (`HasType.mono_env`, `HasType.mono_concept`, `HasType.mono_grant`); each monotonicity is one direction of progressive commitment.

Weakening holds for the delay-free fragment by appending to the context (`HasType.weaken_append`); a stateful term cannot be moved under a binder at all, so no stronger weakening is needed.

4 PROGRESSIVE REALIZATION

A declaration evolves: it is declared with a signature, it acquires commitments, it acquires a realization, its commitments are strengthened. Throughout, other declarations refer to it. This section gives design progression its mathematical form — a *refinement order* generated by three steps — and proves that progression preserves what was established before it: clients typed against a relationship stay typed (Theorem 3), and clients whose commitments were discharged through it stay discharged, provided the validation layer’s evidence is monotone (Theorem 4), a proviso that Theorem 5 shows cannot be dropped.

4.1 Interfaces, evidence and satisfaction

An interface $\mathcal{P} = \langle \tau, \mathcal{K} \rangle$ is the relationship’s public promise: an expected type and a list of commitments — atomic labels such as `total`, `monotone`, `bounded` that a client may rely on. Interfaces are ordered by monotone refinement:

$$\mathcal{P} \sqsubseteq \mathcal{P}' := \mathcal{P}.\tau = \mathcal{P}'.\tau \wedge \mathcal{P}.\mathcal{K} \subseteq \mathcal{P}'.\mathcal{K},$$

a decidable preorder, frozen on the type and growing on commitments (`InterfaceRefines`). Nothing else is an interface refinement.

What discharges a commitment is not the kernel’s business; it is the validation layer’s. The kernel abstracts it as an **evidence** relation $ev \Delta e p$ between a design, a term and a property. Evidence takes the environment because compositional discharge needs it — “ A is monotone because B is committed to be monotone” consults B ’s interface. A realization e **satisfies** $\mathcal{P}$ in Θ, Δ, Γ when it has the expected type under the grant of that type and every commitment is discharged:

$$\text{Satisfies } ev \Theta \Delta \Gamma e \mathcal{P} := \Theta; \Delta; \Gamma \vdash_{\text{grant}(\mathcal{P}, \tau)} e : \mathcal{P}. \tau \wedge \forall p \in \mathcal{P}. \mathcal{K}. ev \Delta e p.$$

A declaration is well formed when its body, if any, satisfies its interface; a design is **globally well formed**, $\text{GlobalWF } ev \Theta \Delta$, when every stored declaration sits under its own identity and is well formed in Δ at top level. An unrealized declaration is always well formed.

The refinement order is complete for abstract evidence: $\mathcal{P} \sqsubseteq \mathcal{P}'$ iff every realization of $\mathcal{P}'$ in every environment under every evidence relation realizes $\mathcal{P}$ ($\text{InterfaceRefines_iff_semantic}$). The proof of the converse instantiates evidence at “the property is in $\mathcal{P}'$ ’s list” and the body at a reference to a single declaration.

4.2 The refinement order and the lifecycle

Design progression is generated by three steps (DeclRefines), each preserving the identity by construction and each checked against the current environment Δ :

$$\frac{\mathcal{P} \sqsubseteq \mathcal{P}'}{\langle \delta, \mathcal{P}, \text{none} \rangle \rightsquigarrow \langle \delta, \mathcal{P}', \text{none} \rangle} \quad \frac{\text{Satisfies } ev \Theta \Delta \Gamma e \mathcal{P}}{\langle \delta, \mathcal{P}, \text{none} \rangle \rightsquigarrow \langle \delta, \mathcal{P}, \text{some } e \rangle} \quad \frac{\mathcal{P} \sqsubseteq \mathcal{P}' \quad \text{Satisfies } ev \Theta \Delta \Gamma e \mathcal{P}'}{\langle \delta, \mathcal{P}, \text{some } e \rangle \rightsquigarrow \langle \delta, \mathcal{P}', \text{some } e \rangle}$$

An unrealized declaration may have its interface refined; an unrealized declaration may be realized by a satisfying computation; a realized declaration may have its interface strengthened provided the realization is *re-verified* against the new interface. Strengthening without re-verification breaks well-formedness, and the counterexample is mechanized ($\text{naive_breaks_wellformedness}$).

Separately from the steps there is a purely structural order with no satisfaction condition: $\text{DeclLeq } h h'$ requires the same name, $h.\mathcal{P} \sqsubseteq h'.\mathcal{P}$, and a write-once definiens (if h has definiens e then so has h'); the pointwise lifting $\Delta \sqsubseteq \Delta'$ (EnvRefines) permits new declarations besides. Storing a refined declaration back under its name is an environment refinement: if Δ holds h at its name and $\text{DeclLeq } h h'$, then $\Delta \sqsubseteq \Delta[h']$ (EnvRefines_update). This is the one place identity does any work: it makes the update land on the slot every reference resolves to, which is what a name does in any environment semantics.

PROPOSITION 2 (The lifecycle is the structural order; $\text{DeclRefinesStar_iff}$). *The reflexive–transitive closure of the three steps, all side conditions checked in Δ , relates h to h' iff $\text{DeclLeq } h h'$ and h' is well formed in Δ .*

Proof. ($\Rightarrow$) Each step preserves the identity, refines the interface (the type unchanged, the commitment list extended) and never removes a definiens — so each step is below DeclLeq , which is transitive, and each step’s side condition is exactly that the target is well formed. ($\Leftarrow$) Given $\text{DeclLeq } h h'$ and h' well formed, write $h = \langle \delta, \mathcal{P}, r \rangle$ and $h' = \langle \delta, \mathcal{P}', r' \rangle$ with $\mathcal{P} \sqsubseteq \mathcal{P}'$. If $r = \text{none}$ and $r' = \text{none}$, one refine step suffices. If $r = \text{none}$ and $r' = \text{some } e$, take a refine to $\langle \delta, \mathcal{P}', \text{none} \rangle$ followed by a realize with e , whose side condition — e satisfies $\mathcal{P}'$ — is the well-formedness of h' . If $r = \text{some } e$ then $r' = \text{some } e$ (a definiens is never dropped), and one strengthen step to $\mathcal{P}'$ suffices, its side condition again being the well-formedness of h' . $\square$

4.3 Client stability

Another part of the product may already depend on a relationship before that relationship is realized (§2.2). Can the relationship then be realized, or strengthened, without editing those clients and without invalidating what was established about them? The answer has two halves with deliberately different hypotheses, and together they are the paper’s central result: progress in the design does not destroy the meaning of earlier design decisions.

THEOREM 3 (Clients survive realization — typing; $\text{local_refinement_preserves_global_typing}$). *Let B be declared in Δ with record h , and let h' be above h in the structural order, $\text{DeclLeq } h h'$. Then every judgment $\Theta; \Delta; \Gamma \vdash_G e : \tau$ holds in $\Delta[h']$, the design with B ’s record replaced by h' .*

Proof. Let $\Delta' = \Delta[h']$. For every constant δ' , the type view agrees: if $\delta' \neq B$ then $\Delta' \delta' = \Delta \delta'$, and if $\delta' = B$ then $\Delta' \ni B : \tau$ exactly when $\Delta \ni B : \tau$, because $\text{DeclLeq } h h'$ keeps the expected type. Now proceed by induction on the derivation of $\Theta; \Delta; \Gamma \vdash_G e : \tau$: T-Ref is the only rule that reads Δ , and it reads the type view, which is the same in Δ' ; every other rule is reproduced with the induction hypotheses. (This is the monotonicity lemma HasType.mono_env : typing is preserved by any change of Δ that preserves the type view.) $\square$

The proof is one line: typing reads Δ through the type view, and the type view is invariant under DeclLeq. That the proof is short is the point, not a weakness. The theorem says that the decision to let clients see a relationship's promise and never its realization is *sufficient* for every client to survive every realization and every added commitment, with no side condition. It is also necessary: change B 's expected type while keeping its identity and every client breaks, which is why the type is frozen in $\sqsubseteq$ and why changing it is an edit (§4.4).

The commitment half needs more.

DEFINITION (Monotone evidence). *An evidence relation ev is **monotone** when, for all designs $\Delta_1 \sqsubseteq \Delta_2$ (pointwise refinement, EnvRefines), every term e and every property p , $ev \Delta_1 e p$ implies $ev \Delta_2 e p$. Evidence that ignores the environment is monotone; evidence that consults only the presence of commitments and realizations is monotone; evidence that consults their absence is not.*

THEOREM 4 (Clients survive realization — commitments; local_refinement_preserves_global_wf, local_lifecycle_preserves_global_wf). *Let ev be monotone, let Δ be globally well formed under ev and Θ , let B be declared in Δ with record h , and let $h \rightsquigarrow h'$ be a lifecycle step whose side conditions are checked in Δ . Then $\Delta[h']$ is globally well formed. The same holds for a whole lifecycle $h \rightsquigarrow^* h'$ checked against the original Δ .*

Proof. Write $\Delta' = \Delta[h']$ and note first that $\Delta \sqsubseteq \Delta'$ pointwise (EnvRefines): every declaration of Δ is below the corresponding one of Δ' , since only B changed and it moved up by DeclLeq. Global well-formedness of Δ' asks, for every δ with $\Delta' \ni \delta := b$, that b satisfies δ 's interface in Δ' . Two cases. If $\delta = B$, the step's side condition gives that h' 's definiens satisfies h' 's interface in Δ ; satisfaction is a typing judgment plus evidence for each commitment, the typing transfers to Δ' by Theorem 3, and the evidence transfers because ev is monotone along $\Delta \sqsubseteq \Delta'$. If $\delta \neq B$, the declaration is unchanged and was well formed in Δ by hypothesis; the same two transfers move that fact to Δ' . For the lifecycle version, induct on the sequence of steps with the invariant “the current environment refines Δ , is globally well formed, and still holds B 's record as the steps expect”: each step is a single-step instance whose side conditions, checked in Δ , transfer to the current environment by monotonicity, and updating twice at one name is updating once. $\square$

THEOREM 5 (Monotonicity is necessary; badEv_not_mono). *There is an evidence relation ev_{bad} , a globally well formed two-declaration design, and a valid realization step of one declaration after which the design is not globally well formed; consequently ev_{bad} is not monotone.*

Proof. Take two declarations $A : \text{Nat} \rightarrow \text{Bool}$, realized and committed to `total`, and $B : \text{Nat} \rightarrow \text{Nat}$, unrealized, with A 's definiens applying B . Let $ev_{\text{bad}} \Delta e p$ hold exactly when p is `total`, e is A 's definiens and B has no definiens in Δ . The two-declaration design is globally well formed under ev_{bad} (a finite check). Realizing B with the identity function is a valid realize step: its side condition is checked in the original environment, where B is unrealized. In the updated environment B has a definiens, so ev_{bad} no longer discharges `total` for A , and the design is not globally well formed. Were ev_{bad} monotone, Theorem 4 would contradict this; hence it is not. $\square$

The relation ev_{bad} discharges “ A is total” whenever the declaration A reads is still unrealized — evidence from absence. Realizing that declaration is a perfectly valid step, and it destroys the discharge. The monotonicity hypothesis was not part of the original design; it appeared when Theorem 4 was attacked, and it is a Kripke-style stability condition imposed by the kernel on the validation layer: any discharge mechanism meant to survive design progression must be positive in the environment. Its design reading is direct — a commitment may be justified by what other relationships promise, never by what they have not yet decided.

Binding a representation to a previously unbound concept is likewise a refinement: typing, satisfaction and global well-formedness are monotone in Θ (GlobalWF.of_conceptRefines). Rebinding a concept to a different representation is an edit that breaks existing realizations (representation_change_is_edit_not_refinement).

4.4 Refinement versus edit

Theorems 3–4 cover refinement only; they are what the calculus promises about design progression, and the line between progression and *edit* is part of the design. Table 2 classifies the operations a tool offers on a declaration B read by a client A ; each row is witnessed by a mechanized example on a two-declaration design.

operation on B	kind	effect on A
add a public commitment	refinement	typing and commitments preserved
realize	refinement	preserved; A now unfolds to a closed program
strengthen a realized interface	refinement, with re-verification	preserved
change the expected type, keep identity	edit	typing broken
drop a commitment	edit	typing silent; A 's commitment broken
replace B by a new identity	edit	dangling reference
detach or replace the realization	edit	evidence that consulted the body is void
assign or change a clock domain (§6)	edit	domain judgment on clients broken
retarget an output binding (§8)	edit	completeness or single-driver may break

Table 2. *Refinement versus edit.*

Two rows are instructive. Dropping a commitment changes no type, so the type checker is silent, yet A 's own commitment was discharged through B 's and is now unsupported: commitments are part of the interface in the same load-bearing sense as the expected type. Detaching a realization is an edit for the same reason: clients' typing is unaffected, but evidence that consulted the body is void. The kernel does not forbid edits; it declines to promise anything about them, and a tool must reopen the validation of transitive dependents.

4.5 Unfolding

Before time enters, the semantics of a design is *unfolding*: replace each reference to a realized declaration by its realization, recursively, stopping at unrealized declarations (Unfolds $\Delta e e'$). Let $\text{DependsOn } \Delta a b$ hold when the body of a refers to b .

PROPOSITION 6 (Unfolding; Unfolds.det, Unfolds.exists_of_acyclic, Unfolds.not_of_cyclic, Unfolds.refFree_of_fullyRealized). *Unfolding is deterministic; on the delay-free fragment it exists iff the reference graph is acyclic; a reference on a cycle through realized declarations has no unfolding at all; and a fully realized well-typed design unfolds to a reference-free program of the same type.*

Proof. Unfolding is the least relation that copies every syntactic form, leaves an unrealized constant in place, and replaces a realized constant by the unfolding of its definiens. *Determinism*: induction on one unfolding derivation, inverting the other; at a constant, the definiens is determined by Δ . *Existence under acyclicity*: an acyclic Δ comes with a rank that strictly decreases along every reference edge; prove, by strong induction on a bound n , that every term whose referenced constants all have rank below n unfolds — the structural cases pass the bound to their subterms, and a realized constant δ of rank below n has a definiens whose references have rank below $\text{rank}(\delta) < n$, so the inner induction hypothesis applies. *No unfolding on a cycle*: an unfolding of δ carries a derivation whose referenced constants are not on a cycle through δ (proved by induction on the derivation: the only way to pass through a realized constant is to unfold it, and the result's references are those of the definiens, so a reference back to δ would give a smaller derivation of the same shape, which is impossible); a constant reaching itself contradicts this. *Reference-freeness*: if every constant is realized, no `refStuck` case can occur, so the result mentions no constant; its type is preserved because each unfolding step replaces a constant by a definiens of the same type, typed under the universal grant. $\square$

Acyclicity is witnessed by a rank that strictly decreases along edges, $\text{Acyclic } \Delta := \exists \text{rank} . \forall a b . \text{DependsOn } \Delta a b \rightarrow \text{rank } b < \text{rank } a$, and excludes cycles ($\text{Acyclic.not_cyclic}$). The pure fragment has no fixpoints, so a cyclic definition denotes nothing; §6 shows which cycles become meaningful once delay exists, and that unfolding agrees with tick evaluation on the first-order fragment (`unfolds_preserves_eval`).

5 SEMANTIC INTEGRITY

A relationship connects *meanings*. `dimByTilt : Tilt -> Brightness` relates a product concept to a product concept, and it is not the relationship `MotorAngle -> Brightness` even though a tilt and a motor angle are both angles. Section 4 typed declarations without saying what a type means; this section adds the two things a product concept carries that a number does not — an identity that survives representation, and a physical dimension — and shows that a

realization, when it arrives, is constrained to preserve the semantic boundary the signature drew before it existed. The distinction the section rests on is stated once: a **concept** is what the designer means, its **representation** is the data it is carried by, and the two are bound separately, later, and write-once.

5.1 Nominal identity, and the grant as realization authority

Why not identify concepts by representation? Suppose concepts were represented only by their representation types, so that `Tilt` and `MotorAngle` are both Q_{Angle} . Then the wire `motorTarget := tiltSensor` is well typed and the design is globally well formed, because nothing in the model records the distinction the designer drew. Nominal types C over an internal identity record it: two distinct identities are distinct types regardless of representation, so the invalid wire is rejected by T-App with no additional judgment. An explicit relationship `tiltToMotor : Tilt -> MotorAngle` is an ordinary declaration of arrow type — signature-first, possibly unrealized — and it appears in the term wherever a crossing occurs. The kernel has no cast, coercion or conversion.

Why not let any realization construct any concept of matching representation? Nominal identity alone leaves concept values opaque: under T-Ref and T-App only, a value of C can originate only in a declaration of concept type (`no_semantic_value_without_declaration`). That is the right state *before* a realization exists. To let a formula realize a relationship, representation must be observable and constructible, and the obvious way to add it destroys what identity has bought. With global $\text{rep}_C : C \rightarrow R$ and $\text{mk}_C : R \rightarrow C$ available everywhere, $\lambda x. \text{mk}_{\text{Motor}}(\text{rep}_{\text{Tilt}} x)$ is a well-typed `Tilt -> MotorAngle` in the empty environment with no declared relationship (`unrestricted_representation_binding_bypasses_semantic_identity`), and the crossing can hide inside a body whose signature mentions no motor (`hidden_crossing_inside_unrelated_body`). Observation alone is safe but cannot realize a mapping.

The grant separates the two, and its design reading is *realization authority*: the signature the designer wrote before any computation existed is what authorizes the computation's result. Observation `rep` is typed everywhere (T-Rep); construction `mkC` is typed only where $C \in G$ (T-Mk); client code is typed under $\emptyset$ and a realization of type τ under $\text{grant}(\tau)$, the grant of its own signature (the definition of Satisfies). A realization of `Tilt -> Brightness` may construct a `Brightness` and nothing else — not a `MotorAngle`, not an `Opacity`, whatever their representations. Let $e.$ constructs C hold when `mkC` occurs in $e.$

THEOREM 7 (Realization authority; `HasType.constructs_granted`). *If $\Theta; \Delta; \Gamma \vdash_G e : \tau$ and $e.$ constructs C , then $C \in G$. Under $\text{grant}(\tau)$: a value of C is built only inside a realization whose signature announces C .*

Proof. By induction on the typing derivation, with $e.$ constructs C meaning that `mkC` occurs somewhere in $e.$ The only rule that introduces `mkC` is T-Mk, whose premise is $C \in G$; every other rule leaves the grant unchanged for its subterms, so an occurrence inside a subterm is handled by the induction hypothesis at the same G . Under $G = \text{grant}(\tau)$ for a definiens of type τ , $C \in \text{grant}(\tau)$ says that C is the result concept of τ , i.e. that the signature announces it. $\square$

The hidden crossing above is rejected under the grant of an unrelated declaration and becomes legal, and visible, once `tiltToMotor` is declared (`hidden_crossing_rejected_under_grant`, `representation_binding_does_not_enable_hidden_semantic_mapping`).

The running example, steps 3 and 4 of Table 1. The designer binds $\Theta(\text{Tilt}) = Q_{\text{Angle}}$ and $\Theta(\text{Brightness}) = Q_0$, and realizes δ_1 with $\lambda x : \text{Tilt}. \text{mk}_{\text{Brightness}}(\text{rep } x \cdot g)$ for a gain $g : Q_0 - \text{Angle}$. The body is typed under $\text{grant}(\text{Tilt} \rightarrow \text{Brightness}) = \{\text{Brightness}\}$: T-Rep observes the tilt at Q_{Angle} , the product has dimension 0, and T-Mk constructs the brightness because `Brightness` is in the grant. The same body with `mkMotorAngle` has no derivation, and by Theorem 3 `light`'s typing from §2.2 is untouched by the realization.

Two constraints on representations in Θ . WF were not anticipated. Representations must be concept-free: if `Tilt` may be represented by `MotorAngle`, then `rep` itself is a hidden mapping under every policy including observation-only. And they must be data, a requirement that arrived from the reactive semantics: a concept value may be delayed, and a function-typed representation would carry a closure across ticks.

The grant is a known shape — the private constructor of an abstract type exported only to its defining module [4], or a capability attached to a definition site. What is specific is that the capability comes from the signature the designer already wrote, so no annotation is added, and one consequence follows: after all bodies are inlined

into one program, that program is checked under the universal grant, because each construction was authorized at its own declaration. Semantic isolation is a property of the design graph and survives inlining as provenance (Theorem 12), not as a type property of the executable.

5.2 Representation is not meaning: erasure

Let $\eta : \text{ConceptId} \rightarrow \text{Ty}$ map each concept to a data type, agreeing with Θ on bound concepts. Erasure τ^η replaces each concept C by $\eta(C)$ throughout a type; on terms, $\text{rep } e$ and $\text{mk}_C e$ erase to e^η , and the type indices of operators are erased.

PROPOSITION 8 (Erasure is sound; `HasType.erase`). *If Θ .WF, η agrees with Θ , and $\Theta; \Delta; \Gamma \vdash_G e : \tau$, then $\Theta; \Delta^\eta; \Gamma^\eta \vdash_{G'} e^\eta : \tau^\eta$ for every grant G' .*

Proof. By induction on the derivation. Erasure maps each concept C to $\eta(C)$, agreeing with Θ where the latter is defined, and replaces $\text{mk}_C e$ by e^η and $\text{rep } e$ by e^η . In the T-Mk case the premise types e at $\Theta(C) = R$; since Θ is well formed, R is concept-free, so $R^\eta = R = \eta(C) = C^\eta$, and the erased term e^η has the erased type. The T-Rep case is symmetric. T-Ref uses the erased type view Δ^η , and no rule needs the grant after erasure, so any G' serves. All remaining rules commute with erasure syntactically. $\square$

Erasure is not injective — `Tilt` and `MotorAngle` erase to the same type (`erase_not_injective`) — and the untyped baseline is exactly what erasure leaves: the design the nominal calculus rejects is accepted after erasure (`baseline_is_erased_modelA`). Generated code is therefore ordinary code; the semantic layer has no runtime residue. This is the precise sense in which representation is not meaning: the meaning lives in the design’s declarations and is checked there, and the representation is all that runs.

Three alternatives were formalized and refuted. A model in which the display name *is* the identity makes renaming destructive (`rename_under_name_identity_breaks_client`), whereas here a rename preserves identity (`semantic_rename_preserves_identity`). Treating a concept as an ordinary declaration admits category errors: the concept becomes usable as a value and can be realized by a number. Keeping identity out of the type as interface metadata checked by a direct-wire rule is evaded by η -expansion, since $(\lambda x. x) \text{ tilt}$ has the same flow with no direct wire (`bweak_evaded_by_eta`); a compositional role judgment strong enough to close that gap has the rule shapes of typing over the concept types and duplicates it.

5.3 Dimensions: coherent arithmetic on representations

Dimensions play a narrower role than concept identity. Once a concept is observed through `rep`, the arithmetic on its representation must remain physically coherent, and that is all dimensions do. A physical quantity has type Q_d . There is no dimension-specific typing rule: $\text{add}_d : Q_d \rightarrow Q_d \rightarrow Q_d$, $\text{mul}_{d_1 d_2} : Q_{d_1} \rightarrow Q_{d_2} \rightarrow Q_{d_1+d_2}$ and $\text{div}_{d_1 d_2}$ with $d_1 - d_2$ are registered operators, and an application is checked by T-App. `length + time` is ill typed (`dimension_mismatch_rejected`); erasing every dimension to the zero vector is a sound translation that accepts it (`counterexampleB_baseline_accepts_length_plus_time`), so the untyped numeric baseline is the erasure of dimensional typing in the same sense that it is the erasure of nominal typing.

Dimension and identity are orthogonal, and the orthogonality is what the relationship-first reading needs: `Tilt` and `MotorAngle` both bound to Q_{Angle} remain distinct types (`same_dimension_does_not_imply_same_semantic_identity`); a relationship realized by the dimensioned formula $\lambda x. \text{mk Brightness } (\text{rep } x \cdot \text{gain})$ with $\text{gain} : Q_{0-\text{Angle}}$ is typed, a dimension error inside it is caught by the same typing, and the formula cannot manufacture a `MotorAngle` despite the shared dimension (`explicit_semantic_mapping_uses_dimensioned_formula`). The association between a concept and its dimension lives in Θ , not in the identity and not in the type constructor: the designer says *tilt to brightness* first and *tilt is an angle* separately.

Units are not in the calculus at all. A literal `90 deg` elaborates to $\text{lit}_{\text{Angle}}$ of a scaled magnitude; a coordinate $\text{inUnit}(q, u)$ is q divided by a scale constant and has dimension zero; $\text{withUnit}(x, u)$ is the converse; a conversion is their composition. Each is elaboration, none is a kernel construct, and a unit choice never reaches a type: `1 m` and `100 cm` are equal values of one type. The unit laws are proved above the kernel over an abstract scalar domain and instantiated exactly by a symbolic group in which π is a generator, so that a degree is exactly $\pi/180$ radian; we do not develop them here. Dimensional typing is thus Kennedy’s discipline [6], [7] without unit polymorphism in

the kernel: dimension variables appear only in the surface’s definitional families (§7.2), where matching against closed dimensions instantiates them.

6 RELATIONSHIPS IN TIME

A relationship in an interactive physical product does not hold only in a type space; it holds *over time*, and it holds in an authored temporal structure — the tilt moves with the interaction, the room temperature with the environment. This section interprets declarations in time. Its order follows the thesis: first what it means for a declared relationship to have a value at a tick (a stream, with unrealized declarations read from the environment), then how memory enters a relationship, then which timing domain a relationship’s value belongs to, and finally why the one rule that governs reading across domains is the one that keeps the designer’s temporal structure intact.

6.1 Declarations as streams

Values are booleans, naturals (which also carry every Q_d ; the executable kernel’s magnitudes are naturals), tagged concept values $\text{mk}_C v$, none , $\text{some } v$, lists, pairs, closures $\text{clo } \rho e$ over a value environment, and partially applied operators $\text{prim } p \vec{v}$. An operator is computed when saturated: $\text{applyPrim } p \vec{v}$ is compute $p \vec{v}$ if $|\vec{v}|$ equals p ’s arity and $\text{prim } p \vec{v}$ otherwise.

The judgment $\rho \vdash_t e \Downarrow v$ — the value of e at tick t under local environment ρ , with the design Δ and the input I ambient — is defined in Figure 4 (Ev).

$$\begin{array}{c}
\frac{\rho(x) = v}{\rho \vdash_t x \Downarrow v} \quad \frac{}{\rho \vdash_t \lambda x : \tau. e \Downarrow \text{clo } \rho e} \quad \frac{}{\rho \vdash_t p \Downarrow \text{applyPrim } p []} \\
\frac{\rho \vdash_t f \Downarrow \text{clo } \rho' b \quad \rho \vdash_t a \Downarrow w \quad w :: \rho' \vdash_t b \Downarrow v}{\rho \vdash_t f a \Downarrow v} \quad \frac{\rho \vdash_t f \Downarrow \text{prim } p \vec{u} \quad \rho \vdash_t a \Downarrow w}{\rho \vdash_t f a \Downarrow \text{applyPrim } p (\vec{u} ++ [w])} \\
\frac{\Delta \ni \delta := b \quad [] \vdash_t b \Downarrow v}{\rho \vdash_t \delta \Downarrow v} \text{ (E-Real)} \quad \frac{\delta \notin \text{def}(\Delta)}{\rho \vdash_t \delta \Downarrow I \delta t} \text{ (E-Input)} \\
\frac{\rho \vdash_t e \Downarrow \text{mk}_C w}{\rho \vdash_t \text{rep } e \Downarrow w} \quad \frac{\rho \vdash_t e \Downarrow w}{\rho \vdash_t \text{mk}_C e \Downarrow \text{mk}_C w} \\
\frac{\rho \vdash_0 i \Downarrow v}{\rho \vdash_0 \text{delay } i e \Downarrow v} \text{ (E-Delay0)} \quad \frac{\rho \vdash_t e \Downarrow v}{\rho \vdash_{t+1} \text{delay } i e \Downarrow v} \text{ (E-DelayS)} \\
\frac{\rho \vdash_t f \Downarrow v_f \quad \rho \vdash_t z \Downarrow v_z \quad \rho \vdash_t l \Downarrow \text{list } []}{\rho \vdash_t \text{fold } f z l \Downarrow v_z} \text{ (E-FoldNil)} \\
\frac{\rho \vdash_t f \Downarrow v_f \quad \rho \vdash_t z \Downarrow v_z \quad \rho \vdash_t l \Downarrow \text{list } (x :: xs) \quad [\text{list } xs, v_z, v_f] \vdash_t \text{fold } \#2 \#1 \#0 \Downarrow r \quad [r, x, v_f] \vdash_t \#2 \#1 \#0 \Downarrow v}{\rho \vdash_t \text{fold } f z l \Downarrow v} \text{ (E-FoldCons)}
\end{array}$$

Figure 4. Single-domain evaluation (Ev), with Δ and I ambient. In one domain sync_κ evaluates exactly as delay (rules syncZero , syncSucc), which §6.7 justifies. Literals evaluate to themselves. The variable $\#i$ is de Bruijn index i .

Three points of Figure 4 deserve comment. An unrealized declaration is an *input*: E-Input reads $I \delta t$, the environment’s realization of the relationship. A realized declaration is evaluated from its realization at the current tick in the *empty* environment (E-Real): a reference’s value never depends on the local environment of the reader, which is what makes a relationship a stream the design observes rather than a function of its call site (Ev.declRef_env_irrelevant). And delay shifts the tick: read at $t + 1$, it evaluates its operand at t ; at 0 it evaluates the initial value.

The recursor’s rule unrolls syntactically. Rather than a recursive definition of a fold on values, the rule evaluates the syntactic term $\text{fold } \#2 \#1 \#0$ in an environment holding the tail, the seed and the function, and then the term $\#2 \#1 \#0$ in an environment holding the result, the head and the function. This keeps Ev

an ordinary inductive relation with no mutual recursion, so every proof by induction on Ev that predated the recursor extends by one case, and totality is a separate lemma by induction on the list (§7.1).

THEOREM 9 (Determinism; Ev.det). *If $\rho \vdash_t e \Downarrow v_1$ and $\rho \vdash_t e \Downarrow v_2$ then $v_1 = v_2$.*

Proof. By induction on the first derivation, inverting the second: each syntactic form has at most one applicable rule per tick (for `delay` and `sync` the tick decides between the initial and the successor rule, and for a constant the presence of a definiens decides between `E-Real` and `E-Input`), the environment lookup, the input and the definiens are functions, and the induction hypotheses identify the values of the premises. $\square$

Evaluation is a partial function with no hidden evaluation order — there are no effects to order — and this holds unconditionally.

An executable interpreter `evalF` with a fuel parameter is proved sound for the relation (`evalF_sound`, Ev.of_evalF). Every trace in the development and in this paper was computed by it inside the proof checker.

6.2 Cycles and causality

Let e . `instRefs` be the declarations e refers to *instantaneously*: those not under the delayed operand of a `delay` or `sync` (the initial value is read at tick 0 and counts as instantaneous). The relation `InstDependsOn` $\Delta a b$ holds when b is among the instantaneous references of a 's definiens.

DEFINITION (Causal). A design Δ is **causal** when there are a rank rank on constant names and a bound R with $\text{rank } \delta < R$ for every δ , such that every instantaneous dependency strictly decreases the rank: `InstDependsOn` $\Delta a b$ implies $\text{rank } b < \text{rank } a$. We write **Causal** Δ , and say that rank, R witness it.

On the delay-free fragment `InstDependsOn` is `DependsOn`, so causality is exactly bounded acyclicity (`Causal_iff_acyclic_of_delayFree`): the earlier condition is the timeless special case rather than a replaced requirement. A structural cycle every path of which passes through a delayed operand — $A := \text{delay } \emptyset B$; $B := A$, or a self-delayed accumulator — is causal. A cycle that is partly delayed is not.

PROPOSITION 10 (Strict cycles have no value; $\text{Ev.not_of_strictCyclic}$). *If a constant δ lies on a cycle of references passing through neither a delayed operand nor a lambda, then for every tick t and environment ρ there is no v with $\rho \vdash_t \delta \Downarrow v$.*

Proof. Every derivation of $\rho \vdash_t e \Downarrow v$ carries the invariant that no *strict* reference of e — a reference not under a delayed operand or a lambda — lies on a strict cycle: `E-Real` passes from δ to the strict references of its definiens, so a strict path from δ back to δ would yield a strictly smaller derivation of the same conclusion, which is impossible by induction; the delay rules read their operand at an earlier tick and do not count as strict; a lambda's body is not evaluated. A constant on a strict cycle violates the invariant at the root. $\square$

Not “some default”, not “one of several”: no derivation exists. A gap should be recorded. A cycle guarded by a lambda, $A := \lambda x. A \ x$, is rejected by **Causal** yet `declRef A` does evaluate — to a closure; only applying it diverges. The judgment **Causal** is conservative for lambda-guarded cycles, and Proposition 10 covers strict cycles only.

6.3 The logical relation and totality

Totality is proved by a logical relation indexed by the tick. The relation is stated generically in an *application relation* $A : \text{Value} \rightarrow \text{Value} \rightarrow \text{Value} \rightarrow \text{Prop}$ — how a function value applied to an argument yields a result — so that the single-domain and multi-domain semantics share one relation. At a fixed tick, A is **Apply** $\Delta I t : v_f$ applied to w yields v when v_f is a closure whose body evaluates to v at t under w , or a partial operator whose saturation is v .

$$\begin{aligned}
\mathcal{R}_\Theta^A[\text{Bool}] v &\Leftrightarrow \exists b. v = \text{Bool } b \\
\mathcal{R}_\Theta^A[\text{Nat}] v &= \mathcal{R}_\Theta^A[Q_d] v \Leftrightarrow \exists n. v = \text{Nat } n \\
\mathcal{R}_\Theta^A[\text{Option } \tau] v &\Leftrightarrow v = \text{none} \vee \exists w. v = \text{some } w \wedge \mathcal{R}_\Theta^A[\tau] w \\
\mathcal{R}_\Theta^A[\text{List } \tau] v &\Leftrightarrow \exists \vec{w}. v = \text{List } \vec{w} \wedge \forall w \in \vec{w}. \mathcal{R}_\Theta^A[\tau] w \\
\mathcal{R}_\Theta^A[\tau \times \sigma] v &\Leftrightarrow \exists x y. v = \text{pair } x y \wedge \mathcal{R}_\Theta^A[\tau] x \wedge \mathcal{R}_\Theta^A[\sigma] y \\
\mathcal{R}_\Theta^A[\tau \rightarrow \sigma] v &\Leftrightarrow \forall w. \mathcal{R}_\Theta^A[\tau] w \rightarrow \exists v'. A v w v' \wedge \mathcal{R}_\Theta^A[\sigma] v' \\
\mathcal{R}_\Theta^A[C] v &\Leftrightarrow \exists w. v = \text{mk}_C w \wedge \forall R. \Theta(C) = R \rightarrow \mathcal{R}^A[R] w
\end{aligned}$$

The concept clause says that a concept value is a tagged representation value. Its inner use of the relation at the representation R is the concept-free relation RedSF ; because Θ . WF makes R concept-free, the definition is well founded on the type without appeal to Θ (Red_semFree). At data types the relation is independent of A (Red_data), which is what allows a delayed value to be transported between ticks. Registered operators are related at their types for any A that saturates them (Red_prim).

Well-typed inputs are inputs related to the type view: $\Delta \ni \delta : \tau \wedge \delta \notin \text{def}(\Delta) \Rightarrow \mathcal{R}_\Theta^{\text{Apply } \Delta \ I \ t}[\tau] (I \ \delta \ t)$ for every t .

THEOREM 11 (Totality under causality; fundamental, reactive_total, Ev.red). *Let Θ . WF, let rank, R witness Causal Δ , let $\text{GlobalWF } \text{ev } \Theta \ \Delta$ and let I be well typed. Then for every tick t , bound r , grant G , and $\Theta; \Delta; \Gamma \vdash_G e : \tau$, and every ρ related to Γ such that every instantaneous reference of e has rank below r , there is v with $\rho \vdash_t e \Downarrow v$ and $\mathcal{R}_\Theta^{\text{Apply } \Delta \ I \ t}[\tau] v$.*

Proof. Let $\mathcal{R}_\Theta[\tau] v$ be the logical relation of §6.3 at the application relation of the single-domain semantics: booleans and numbers are related at Bool , Nat and Q_d ; optionals, lists and pairs componentwise; a value at $\tau \rightarrow \sigma$ is related when applying it to any value related at τ yields a value related at σ ; a value at a concept C is $\text{mk}_C w$ with w related at the representation $\Theta(C)$ through the concept-free relation. Prove the stronger statement: for every tick t and bound r , every well-typed e whose instantaneous references have rank below r evaluates, in every environment related to its context, to a value related to its type. The proof is a triple induction — strong induction on t , inside it strong induction on r , inside that induction on the typing derivation — and the two outer inductions are what the two temporal constructs need.

Constant (T-Ref). If δ has no definiens, its value is the input $I \ \delta \ t$, related by the hypothesis on inputs. Otherwise its definiens b is typed under grant(τ) at top level by global well-formedness; causality gives $\text{rank}(\delta') < \text{rank}(\delta) < r$ for every instantaneous reference δ' of b , so the inner induction hypothesis at bound $\text{rank}(\delta)$ evaluates b in the empty environment, and E-Real lifts the result to δ .

Delay (T-Delay). At tick 0 the initial value's derivation gives the result. At tick $t' + 1$ the operand is evaluated at tick t' by the *outer* induction hypothesis, with the full bound R — a delayed operand may reference anything, since its reference is not instantaneous — and the value is related at τ because τ is a data type, at which the relation does not depend on the application relation (Red_data), so relatedness transports across ticks.

Application (T-App). The function's value is related at $\tau \rightarrow \sigma$ and the argument's at τ ; the definition of the relation at arrow type yields a result value and its relatedness at σ . *Abstraction* produces a closure, related at the arrow type by the induction hypothesis applied to any related argument. *Rep* and *Mk* move between a concept and its representation using the concept clause of the relation; *primitives* are related at their types because each registered operator is total on related arguments (Red_prim); *fold* is an inner induction on the list value.

reactive_total instantiates this at $e = \delta$, bound R , empty environment; Ev.red combines it with determinism to say that the value of a well-typed term is related. ◻

Consequently, in a causal, globally well formed design with well-typed inputs, every declared relationship has a value at every tick, and that value — unique by Theorem 9 — is related to its expected type.

The relation is a step-indexed logical relation in the sense of Appel and McAllester [11] and Ahmed [12], with the tick as the index and the rank as a second, inner index; what differs is that the index counts *time* rather than *steps*, so that the induction on it is the induction that makes a delayed self-reference well defined.

6.4 Two restrictions forced by totality

T-Delay and T-Sync restrict their type to data and their context to empty. Neither restriction was a design decision; each is what the induction of Theorem 11 needs. A delayed closure would be a value at tick t related by $\mathcal{R}^{\text{Apply}} \Delta I^t$ that must be transported to tick $t + 1$, and the arrow clause is tick-indexed and cannot be transported; `Red_data` is exactly the statement that data clauses can. A delay under a binder would evaluate its operand at the previous tick in an environment created at the current one. The restrictions have two corollaries stated as theorems: nothing of function type can be delayed or transported (`arrow_not_delayable`, by inversion), and memory and transport are typed only in the empty context (`delay_not_under_binder`, `sync_not_under_binder`). Temporal state therefore belongs to declarations — a relationship may remember, a function may not — and a reusable stateful behavior is instantiated into fresh declarations (§9) rather than abstracted over.

Initialization is semantic, not validation. Every delay carries an explicit initial value. Two toy relations without one show why: the first tick is either undefined (`first_tick_undefined_without_init`) or nondeterministic (`first_tick_nondeterministic_without_init`).

6.5 Provenance through time

State carries semantic tags; it never creates them. Let v . Taints C hold when the tag C occurs anywhere inside v — including inside closures' environments and bodies.

THEOREM 12 (Semantic integrity over time; `Ev.tag_provenance`, `temporal_state_preserves_semantic_identity`). *If no realization in Δ constructs C , no input value is tainted by C , e does not construct C and ρ is clean, then every value $v \vdash_t e \Downarrow v$ is clean. In particular a delayed value carries exactly the tag of the value delayed.*

Proof. Say a value is *tainted* by C when a `mkC` tag occurs inside it — including inside a closure's environment, and counting a closure whose body could construct C . Prove by induction on the evaluation derivation that if e does not construct C and the environment is clean, the result is clean. The literal, variable and primitive cases are immediate (a registered operator builds no tag). A closure is clean because its body does not construct C and its environment is clean. Application evaluates a clean closure's body in a clean environment. A constant with a definiens is evaluated from that definiens, clean by hypothesis on Δ ; a constant without one yields an input, clean by hypothesis on I . The forms `delay` and `sync` evaluate either the initial value or the operand at an earlier tick, both covered by the induction. Observation `rep` strips a tag, and `mkC'` with $C' \neq C$ adds a foreign one; neither introduces C . The statement about a delayed value carrying exactly the tag of the value delayed is the successor case read directly. $\square$

Combined with Theorem 7 this is the runtime half of semantic integrity: a concept appears in a value only if some signature announces it or some input carries it, at every tick. The typing rule `delay :  $\tau \rightarrow \tau \rightarrow \tau$` at data τ gives the static half — a delayed tilt is a tilt, and a backward difference over a time step has dimension `Length` — `Time` with no derivative primitive.

On the first-order fragment a compiler cares about — *wiring* designs, whose realizations contain no lambda — closures never arise, evaluation is independent of the local environment, and unfolding a reference to its realization preserves the value at every tick (`Ev.noClo`, `Ev.env_irrelevant`, `unfolds_preserves_eval`); *pure* terms, with no reference, memory or transport, have the same value in every design at every tick (`Ev.pure`), which §7.2 uses for the definitional library.

6.6 Clock domains as design context

A clock domain is part of a relationship's design context: it says *when the relationship's value belongs to the design*, and the designer authors it as an identity — “moves with the interaction”, “moves with the environment” — before any rate is known. The time model is one global base tick and a schedule $S : \text{ClockId} \rightarrow \mathbb{N} \rightarrow \text{Bool}$ saying at which global ticks each domain activates. A period n induces the schedule $t \bmod n = 0$ (`Sched.periodic`); the schedule lives outside the design. Domain-local time is not a separate counter but the sequence of a domain's activations. The last activation of κ strictly before t is

$$\text{prevAct } S \ \kappa \ 0 = \text{none}, \quad \text{prevAct } S \ \kappa \ (t + 1) = \text{if } S \ \kappa \ t \text{ then some } t \text{ else prevAct } S \ \kappa \ t,$$

with $\text{prevAct } S \ \kappa \ t = \text{some } t' \Rightarrow t' < t \wedge S \ \kappa \ t'$.

Each declaration is assigned a domain by the clock environment K , or none if it is a domain-agnostic relationship usable anywhere. The clock is interface data in every sense that matters — clients' validity depends on it, it is frozen under refinement, and changing it is an edit (Table 2) — and it is stored as a projection beside the interface, as a concept's representation is stored in Θ rather than in the type.

The **domain judgment** $\text{Clocked } K \ \kappa \ e$, for $\kappa : \text{Option ClockId}$, says that e may be evaluated in domain κ :

$$\text{Clocked } K \ \kappa \ (\delta) \Leftrightarrow K \ \delta = \text{none} \vee K \ \delta = \kappa$$

$$\text{Clocked } K \ (\text{some } \kappa) \ (\text{delay } i \ e) \Leftrightarrow \text{Clocked } K \ (\text{some } \kappa) \ i \wedge \text{Clocked } K \ (\text{some } \kappa) \ e$$

$$\text{Clocked } K \ (\text{some } \kappa) \ (\text{sync}_{\kappa'} \ i \ e) \Leftrightarrow \text{Clocked } K \ (\text{some } \kappa) \ i \wedge \text{Clocked } K \ (\text{some } \kappa') \ e$$

$$\text{Clocked } K \ \text{none} \ (\text{delay } i \ e) \Leftrightarrow \text{False}$$

$$\text{Clocked } K \ \text{none} \ (\text{sync}_{\kappa'} \ i \ e) \Leftrightarrow \text{False}$$

and homomorphically elsewhere. A reference stays in its domain or is agnostic; a delay needs a domain; $\text{sync}_{\kappa'}$ switches the domain of its operand. A design is well clocked when every realization is clocked in its own declaration's domain. Typing is unchanged and blind to domains: the direct wire between two domains at the same value type is well typed and rejected only by Clocked . Placing the domain in the type instead was tried and set aside: every domain-agnostic relationship would then need clock polymorphism (`clocked_type_forces_polymorphism`), and nothing the type rejects is missed by the judgment.

The running example, step 6 of Table 1. The designer places `light` and `tilt` in the domain `fast` and leaves `dimByTilt` agnostic: $K(\delta_3) = K(\delta_2) = \text{some } \text{fast}$, $K(\delta_1) = \text{none}$. The definiens of `light`, $\delta_1 \ \delta_2$, is clocked in `fast`: the reference to δ_1 is admitted because δ_1 is agnostic, and the reference to δ_2 because it is in the same domain. Had `tilt` been placed in another domain, the reference would need `sync` with an initial brightness, and the domain judgment would say so.

6.7 Multi-domain evaluation

The judgment $\rho \vdash_t^\kappa e \Downarrow v$ — in domain κ at global tick t , with S, Δ, I ambient — is Ev with the two temporal rules replaced by four (MEV):

$$\frac{\text{prevAct } S \ \kappa \ t = \text{none} \quad \rho \vdash_t^\kappa i \Downarrow v}{\rho \vdash_t^\kappa \text{delay } i \ e \Downarrow v}$$

$$\frac{\text{prevAct } S \ \kappa \ t = \text{some } t' \quad \rho \vdash_{t'}^\kappa e \Downarrow v}{\rho \vdash_t^\kappa \text{delay } i \ e \Downarrow v}$$

$$\frac{\text{prevAct } S \ \kappa' \ t = \text{none} \quad \rho \vdash_t^\kappa i \Downarrow v}{\rho \vdash_t^\kappa \text{sync}_{\kappa'} \ i \ e \Downarrow v}$$

$$\frac{\text{prevAct } S \ \kappa' \ t = \text{some } t' \quad \rho \vdash_{t'}^{\kappa'} e \Downarrow v}{\rho \vdash_t^\kappa \text{sync}_{\kappa'} \ i \ e \Downarrow v}$$

The form `delay` reads the previous activation of the current domain; $\text{sync}_{\kappa'}$ reads the previous activation of κ' and evaluates its operand *there*, in κ' . All other rules carry κ unchanged.

PROPOSITION 13 (One temporal primitive; `delay_is_sync_own`, `clocked_delay_iff_sync_own`, `single_domain_embedding`). $\rho \vdash_t^\kappa \text{delay } i \ e \Downarrow v$ iff $\rho \vdash_t^\kappa \text{sync}_\kappa \ i \ e \Downarrow v$, and $\text{delay } i \ e$ is clocked in κ iff $\text{sync}_\kappa \ i \ e$ is. Under the always-active schedule, $\rho \vdash_t^\kappa e \Downarrow v$ iff $\rho \vdash_t e \Downarrow v$, for every κ .

Proof. The two multi-domain rules for `delay` read the operand at the previous activation of the current domain κ (or the initial value if there is none), and the two rules for sync_κ read the operand at the previous activation of κ ; with the source domain equal to the current one the premises coincide, so each derivation converts into the other by renaming the rule. The clock judgment agrees for the same reason. The single-domain semantics of §6 is the multi-domain one under the schedule that activates one domain at every tick, since then the previous activation of that domain is always the previous tick. $\square$

The kernel therefore has one temporal primitive — read a domain at its previous activation — and `delay` is notation for its diagonal; a delay in a slow domain reads three global ticks back where a delay in a fast one reads one, with the same syntax. The single-domain semantics of §6.1 is the one-domain special case of this one rather than a replaced machine.

THEOREM 14 (Determinism and totality in every domain; $\text{MEv.det}, \text{mfundamental}, \text{multi_domain_total}$). *Multi-domain evaluation is a partial function, for every schedule. In a causal, globally well formed design with inputs well typed in every domain, every declared relationship has a value in every domain at every tick, related to its expected type.*

Proof. Determinism is the argument of Theorem 9 with the multi-domain rules: the previous activation of a domain at a tick is a function of the schedule, so the two temporal forms are deterministic given determinism at the earlier tick. Totality is the argument of Theorem 11 with the application relation of the multi-domain semantics and one change in the temporal cases: a $\text{sync}_{\kappa'}$ reads its operand at the last activation of κ' strictly before t , a tick $t' < t$, so the outer strong induction on the global tick still applies, and the initial value covers a domain that has not activated yet. The value at the first activation of a domain is therefore the explicit initial value of every transport into it. $\square$

The proof reuses the logical relation of §6.3 with the application relation $\text{MApply } S \Delta I \kappa t$, and the same lexicographic induction: a transport at t evaluates its operand at $t' < t$ under any rank. Causality is the *same* **Causal** Δ : a transport's operand is never instantaneous, so no cross-domain cycle can be. An interpreter mevalF is proved sound (mevalF_sound). Tag provenance holds across domains ($\text{MEv.tag_provenance}$): transport changes timing, not identity, and a crossing from Tilt@fast to Tilt@slow authorizes neither $\text{Tilt} \rightarrow \text{MotorAngle}$ nor $Q_{\text{Length}} \rightarrow Q_{\text{Time}}$, by the typing rule.

6.8 Strictly before: preserving the authored temporal structure

Why not expose scheduler order? A transport sees only source activations strictly before the destination tick. The rule exists to preserve the designer's declared temporal relationship — “heat reads the light as it stood before this tick” — without adding a fact the designer never authored, namely which of two simultaneously active domains the implementation happens to run first. That is a choice with an observable alternative, and the alternative was built.

THEOREM 15 (Same-tick visibility exposes the scheduler; $\text{scheduling_order_observable}$). *Let $\text{MEv}_{\leq}$ be the semantics in which a transport may also see a simultaneously active source, resolved by a priority between domains. There is a two-domain design, a schedule and an input such that two priorities give two different values to the same declaration at the same tick.*

Proof. Take two domains, *fast* and *other*, both active at tick 1 and *other* for the first time; an input x in *other* with $x \ 1 = 1$; and $a := \text{sync}_{\text{other}} \ 0 \ x$ in *fast*. Under $\text{MEv}_{\leq}$ with *other* ordered first the transport sees the simultaneous activation and a evaluates to 1; with *fast* ordered first there is no earlier activation of *other* and a evaluates to the initial value 0. Both are derivations of the same judgment form, differing only in the order parameter; so the semantics depends on the order — which the design never authored. Under strict-before there is no simultaneous case, and a is 0 regardless of order. $\square$

At tick 1 both domains are active for the first time; with the source first the transport delivers the source's current value, with the destination first it delivers the initial value. The strictly-before rule has no such parameter, and Theorem 14 has no order between simultaneously active domains in its statement. Every crossing costs one destination-visible step; “synchronous sub-domains evaluated in one instant” are, in this model, the same domain.

Rate and identity are distinct. A clone of a domain with the identical schedule is a different domain, and a direct wire between them is rejected ($\text{equal_rate_not_same_domain}$); a domain at the same rate but shifted in phase reads different values through a transport. Rate changes are validation-only: they change the induced schedule and the observed values, but no client's well-formedness. This is where the calculus departs from synchronous languages that recover clocks by inference [9], [13]: the domain is authored, because the information needed to infer it — the realization binding — arrives at the point where a designer is least able to make the decision.

7 DERIVED STRUCTURE

The calculus deliberately has no primitive for most of what a designer names. This section records what is in the kernel for computation over data — one recursive, products, equality — and then two negative design results that follow the same method: before adding a primitive, ask whether the behavior is already derivable from

declarations, memory, transport and lists. Every temporal operator of the surface language is, and so is the one construction most likely to be proposed as primitive, the lossless cross-domain window.

7.1 The recursor, products and equality

The recursor fold $f \ z \ l$ is a *term former*, not a registered operator. The kernel has no recursion, deliberately; a total language needs an eliminator for its inductive data, and fold is the one construct that applies a function value in the course of evaluation. Registered operators never apply closures. The alternative of one primitive per collection operation was rejected because a primitive cannot apply a closure and each would need its own evaluation rule; the alternative of bounded unrolling was rejected because lists — the cross-domain window — are unbounded.

The recursor is total on related values (`fold_total`, `mfold_total`), by an induction on the list separate from Theorem 11, which invokes it in its fold case. Every collection operation — `map`, `filter`, `any`, `all`, `contains`, `append`, `sum`, `zip`, and through `toList` the option eliminators — is a definition over fold, and each is proved to compute the mathematical function it names through one general lemma: the recursor computes `List.foldr g` whenever the step closure implements g on the reachable accumulators (`fold_spec`; then `any_spec`, `all_spec`, `map_spec`, `filter_spec`, `min_spec`, `clamp_spec`, ...). Finite quantification is a fold — $\forall x \in xs. Px$ iff `all xs P` evaluates to true (`forall_in_list`, `exists_in_list`) — and a finite-set literal means membership with duplicates irrelevant (`oneOf_mem`, `oneOf_dup_irrelevant`), so there is no Set type and no uniqueness convention.

Products $\tau \times \sigma$ with `pair`, `fst`, `snd` entered the kernel after the Church encoding was tried and refuted twice. A Church pair is an arrow, and arrows are not data: nothing of function type can be delayed or transported (`arrow_not_delayable`), so paired *state* — a delayed reading with its timestamp — needs a data product. And a Church pair used as a first-class value needs rank-2 types: in a toy System F with a rank measure, the type of `fst` on Church pairs has rank 2 (`church_fst_rank`), and in the prenex fragment a pair instantiated at one result type serves only one projection (`church_pair_prenex_one_projection`). Products are value composition only; they are never a component interface or an output bundle (§9 shows what a tuple-returning declaration does to the dependency graph).

Equality eq_{τ}^f is structural equality at every data type — booleans, numbers, none/some, pairs and lists componentwise, concept values by tag and representation — with the proof $h : \tau$. Data carried *in the syntax*. This is the kernel’s only capability evidence: an equality on a function type is unwritable rather than ill typed, which keeps T-Prim unconditional. On first-order values structural equality is equality (`Value.beq_iff`, by a mutual induction over the nested value type).

Order is deliberately not generalized. A first formulation gave $<$ a structural meaning at every data type — booleans, options, pairs and lists lexicographically — and it was formally consistent. An audit rejected it on the grounds that no such order has a design meaning: `mode1 < mode2` would order modes by a constructor tag, `None < Some x` is an artifact. The structural order was deleted and lt_d restored to quantities only. So $\text{Data} \Rightarrow \text{Eq}$ holds (`Cap.eq_iff_data`) but $\text{Eq} \not\Rightarrow \text{Ord}$; order on a *concept* is a surface capability — a concept the designer declared ordered and represented by a quantity compares as lt_d on `rep`, a term the kernel already admits (`lt_only_on_quantities`, `lt_rejected`, `min_mode_rejected`). Enumerations follow the same rule: equality is natural, declaration order is never silently behavioral order.

7.2 Polymorphism by families, and the library as combinators

Five models of polymorphism were compared: a monomorphic kernel; per-type duplication; rank-1 parametric polymorphism; System F; higher rank. The one adopted is rank-1 *as definitional families*: every library entry is a function $Ty \rightarrow Expr$ (or $Dim \rightarrow Expr$) in the metalanguage, and a scheme is a pattern over type and dimension variables with capability constraints. The kernel sees only the instances (`instances_are_monomorphic`: three uses of `min` are three kernel terms), and T-Prim, T-App and Proposition 1 are unchanged.

Why this needs no kernel support: a use site always has *closed* argument types. Every declaration’s expected type is frozen and closed, and inference is bottom-up, so finding the instance of a scheme is one-way *matching* of the scheme’s pattern against closed types — decidable, returning the unique substitution on the pattern’s variables (`matchTy_sound`, `matchTy_complete`). There is no unification of two open types, no let-generalization inside expressions [14], and no principal-type search; those problems arise when a definition’s type is inferred from

its body, and every definition here carries its signature. The situation is that of local type inference [15] with no bidirectionality needed. Capability constraints are checked after matching (`Scheme.instantiate_sound`), and the two failure points have designer-level explanations: *no instance* and *capability failed*. System F was rejected by measuring what it would add — the prenex fragment *is* instantiation of families — and every candidate higher-rank use has a rank-1 replacement (`applyBoth_rank`, `applyBoth_replacement`). Dimension polymorphism (`sum : list (q d) → q d`) uses the same mechanism with dimension pattern variables; no kind system, because the dimension algebra already lives in the operator table.

Nominality survives all of it. Any family typed at $\alpha \rightarrow \alpha \rightarrow \alpha$, instantiated at concept C , rejects an argument of concept $C' \neq C$, the representations never consulted (`generic_preserves_identity`); the same for Q_d versus Q'_d (`generic_preserves_dimension`). This is Reynolds’s abstraction [5] and Wadler’s free theorems [16] at the level of syntax: a family cannot inspect what it is instantiated at, because it is instantiated by substitution into a closed term.

Every library entry is a **combinator**: variables, literals, lambdas, applications, registered operators, the recursor and `rep` — no reference, no state, no transport, no `mk`. For combinators four facts are proved once and combine into an inlining statement (`lib_expansion`): typing is independent of the design and the grant and reads Θ only through write-once bindings (`HasType.comb_irrelevant`); the value is the same in every design at every tick under every input (`lib_eval_context_free`, from `Ev.pure`); the term is clocked in every domain (`lib_clocked`); nothing is constructed (`Comb.noConstruct`). This is what lets an implementation inline an equation at each use without creating a declaration — a library entry as a declaration would be monomorphic and would enter the dependency graph.

The expressiveness ceiling, stated once: total first-order-data computation over booleans, quantities, concepts, options, lists and pairs, with higher-order functions and one list recursor; generic definitions instantiated at closed types; no general recursion, no type abstraction in terms, no sums (an enumeration with a payload is encoded as a tag paired with an optional payload, and a kernel sum would cost one more eliminator term former exactly like `fold`), no unbounded quantification. This is a design conclusion backed by executed cases and the proved library; it is not a minimality theorem.

7.3 The window: a negative design result

Within one domain an occurrence is a stream of optional type (§7.4). Across domains this fails: `sync` is a zero-order hold, so a slow consumer of a fast event source sees the last value only. Two fast events at ticks 1 and 2 and one event at tick 2 are indistinguishable at the slow activation at tick 3, and a single event at tick 1 followed by a quiet fast tick is dropped outright (`opt_loses_multiplicity_under_sync`). The counterexample is against `sync` as an *event transport*, not against optional types; it says that multiplicity and order are observable across domains and that keeping them requires buffering.

What the destination should see is the source’s activations since the destination’s own previous activation — the *window*, `windowTicks S src dst t`, the source ticks in $[\text{prevAct } S \text{ dst } t, t)$. The window equals the source’s accumulated log read at the current tick minus its length at the previous destination activation (`buffer_from_log_and_cursor`): two single-instant reads, a `sync` of a source-side accumulator and a delay of a cursor. With list data this is five ordinary declarations:

```
log    @src : cons src (delay nil log)      -- source-side accumulator
logD   @dst : sync src nil log             -- the log, transported
seen   @dst : length logD                  -- log length now
cursor @dst : delay 0 seen                 -- log length at the previous activation
window @dst : reverse (take (seen - cursor) logD) -- the new entries, oldest first
```

THEOREM 16 (The window is derivable; `buffer_window_correspondence`). *For every schedule, input, destination domain and tick, if the five declarations are realized as above and src is an input, then $[\] \vdash_t^{\text{dst}} \text{window} \Downarrow \text{list } (\text{map } (I \text{ src}) (\text{windowTicks } S \text{ src dst } t))$.*

Proof. The five declarations are: a log of every source activation (a list in the source domain, extended by one element at each activation), a count of source activations, the count as seen at the destination’s previous activation, the difference of the two, and the window, which takes that many elements from the reversed log. Prove three equations by induction on the tick, each in its domain: the log at tick t is the list of the source’s

values at the source activations up to t ; the count is its length; and the count transported into the destination domain is the length at the destination's previous activation. The window's definiens then evaluates, by the list operators' equations, to the reversal of the first (length now minus length then) elements of the reversed log — that is, the source values at exactly the source activations since the destination's previous activation, in order and with multiplicity, which is the definition of `windowTicks`. No hypothesis on the schedules is used; at a non-activation tick the equations hold with the previous values. $\square$

The elaboration is well typed and well clocked (`buffer_elaboration_well_typed`, `buffer_elaboration_well_clocked`); the list is injective on windows, ordered by tick, and multiplicity-preserving for every predicate (`buffer_lossless`, `window_to_list_preserves_order`, `window_to_list_preserves_multiplicity`). The general negative result is that any summary depending only on the newest k entries, for any fixed k , identifies a k -entry window with a $(k + 1)$ -entry window (`bounded_summary_not_lossless`); a lossless summary is injective and therefore unbounded (`lossless_iff_injective`). Capacity is thus a deployment obligation on the schedule — for periodic schedules one destination period of source activations suffices (`periodic_capacity_sufficient`) — and the only overflow policy that preserves the semantics is to reject the deployment; dropping is a semantic change (`bounded_buffer_agrees`, `negE`).

The point for the calculus is what was *not* added, and the method by which it was not added — a candidate primitive was formalized, its behavior was derived from the existing kernel, and the derivation was proved correct: no event type, no buffer primitive, no scheduler order, no same-tick visibility, no implicit overflow rule. An event stream is a data-typed declaration in a domain; an occurrence is its value at an activation; a lossless view of it across domains is the five declarations; `latest`, `count`, `coalesce` are ordinary computations over window.

7.4 Derived temporal operators

Every temporal operator a surface language offers reduces to `delay` and registered operators. Table 2 lists the elaborations; each is a declaration referring to itself — a self-delayed cycle, the class that structural acyclicity forbade and causality licenses — and each was typed, checked causal, and run on a concrete trace. There is no independent kernel definition of `count` for the reduction to be proved equal to; the claim is that the elaboration has the intended trace.

surface	declaration body
<code>previous x</code>	<code>delay init x</code>
<code>previous x</code> without an initial value	<code>delay none (some x)</code> ; the absence is pushed to consumers
<code>hold init e</code>	<code>getD e (delay init self)</code>
<code>count e</code>	<code>ite (isSome e) (1 + delay 0 self) (delay 0 self)</code>
<code>since e</code>	<code>ite (isSome e) 0 (1 + delay 0 self)</code>
<code>once e</code>	<code>delay false self $\vee$ isSome e</code>
<code>every n</code>	a modulo- n counter over <code>delay</code>
<code>rise b</code>	<code>b $\wedge$ $\neg$ delay false b</code> , as an optional Boolean

Table 3. Derived temporal operators (*Experiments/ReactiveAlternatives.lean*).

There is no signal type in Ty: under this semantics a signal type would be inhabited by exactly the terms of the underlying type and would reject nothing. There is no event type: within one domain an input delivers at most one value per tick by construction, so an occurrence is a stream of optional type, and the streams of type `Option  $\tau$` are exactly the streams of multiplicity at most one. What separates an occurrence from an optional value can only be seen when a source ticks faster than its observer, which is the cross-domain question of §7.3. State has no identity of its own: a cell is a `delay` in a declaration body, consumers refer to the declaration, and there is consequently no notion of two writers to one cell.

8 PHYSICAL EFFECT

A relationship, realized and evaluated, yields a value; a value is not yet an effect. The chain the calculus draws is *relationship* $\rightarrow$ *realized value* $\rightarrow$ *explicit drive edge* $\rightarrow$ *physical effect*, and this section is the last arrow. A declaration computes a value; it does not move hardware. Physical effect happens only through an explicit **drive edge** from a declaration to a nominally identified **output** — a logical actuator channel, a resource in a different

sort from both concepts and declarations: “the desired steering angle” — a declaration of concept type, one instance of the concept — is a value; “the steering motor” is a resource.

- $\Omega : \text{OutputId} \rightarrow \text{Option } \langle \text{accepts}, \text{clock} \rangle$ — each output’s accepted type and domain;
- $\beta : \text{DeclId} \rightarrow \text{Option OutputId}$ — the drive edges, a write-once per-declaration projection of the same shape as K ;
- $\text{DriveWF } \Omega \ K \ \Delta \ \beta := \forall \delta \ o. \ \beta \ \delta = \text{some } o \rightarrow \exists \ \text{spec} . \ \Omega \ o = \text{some } \text{spec} \wedge \Delta \ni \delta : \text{spec} . \text{accepts} \wedge K \ \delta = \text{some } \text{spec} . \text{clock}$;
- $\text{SingleDriver } \beta := \forall \delta_1 \ \delta_2 \ o. \ \beta \ \delta_1 = \text{some } o \rightarrow \beta \ \delta_2 = \text{some } o \rightarrow \delta_1 = \delta_2$;
- $\text{CompleteOutputs } \beta \ \text{req}$ — every required output is driven.

Nothing was added to types, typing, the domain judgment, evaluation or the grant. The edge neither coerces nor converts nor synchronizes: the driver’s type *equals* the accepted type and its domain is the output’s. A declaration typed `Tilt` cannot drive a `MotorAngle` output; an output that accepts a representation type needs an explicit `rep`-typed declaration in front of it; a slow driver reading a fast value must sync it upstream. A driver of a concept-accepting output is necessarily a value, not a function (`driver_is_unit_domain`).

Definition. $\text{PhysicalOutput } S \ \Delta \ I \ \Omega \ \beta \ o \ t \ v := \exists \delta \ \text{spec} . \ \beta \ \delta = \text{some } o \wedge \Omega \ o = \text{some } \text{spec} \wedge [] \vdash_t^{\text{spec} . \text{clock}} \delta \Downarrow v.$

THEOREM 17 (One driver, one output; `single_driver_output_deterministic`, `multiple_direct_drivers_rejected`). *Under `SingleDriver` β , `PhysicalOutput` is a partial function of o and t . Two declarations driving one output — each well typed, well clocked, causal and individually well formed — violate `SingleDriver` and nothing else, and there is a tick at which the output receives two values.*

Proof. A physical output value at o and t is the value of some driver δ with $\beta \ \delta = o$, evaluated in the domain the output specifies. Given two such values from drivers δ_1 and δ_2 , `SingleDriver` β gives $\delta_1 = \delta_2$, the specification of o is unique, and Theorem 14 identifies the two values. The rejection of two direct drivers is a finite check on the design of Counterexample A: both edges pass typing, clocks, causality and the per-edge condition, and only the global count fails. $\square$

The running example, step 7 of Table 1. The designer declares the output $\Omega(\text{led}) = \langle \text{Brightness}, \text{fast} \rangle$ and draws the drive edge $\beta(\delta_3) = \text{led}$. The edge is well formed because `light` has exactly the accepted type and lives in exactly the output’s domain; `SingleDriver` holds because no other edge targets `led`; and by Theorem 17 the LED’s value at each tick is `light`’s. Drawing a second edge from `dimByTilt` is refused by the type (an arrow is not a `Brightness`), and a second edge from another `Brightness`-typed constant by the invariant.

Why not hide output arbitration? The principle is *many contributors, one explicit final driver*. Contributors are dependencies: `base + corr -> final -> motor` passes every check; priority is a conditional in the single driver; `blend`, `maximum` and `clamp` are ordinary declarations of the target type. Why arbitration must be explicit is shown rather than argued: first-wins, last-wins and maximum over the same value graph give three different physical outputs (`hidden_arbitration_observable`). Binding an unbound declaration to an undriven output is a refinement and preserves `SingleDriver` (`first_output_binding_is_monotone`); binding to a driven output is invalid; retargeting, renaming or detaching an edge invalidates an unchanged design.

Two alternatives were formalized in toy form. Direct effect rows — the set of outputs a declaration drives — are exactly the drive edges, and single-driver is exactly their pairwise disjointness; propagated rows flag a valid design in which a display reads the driver; action values move the conflict into the collector that consumes them, which must then be a policy, which is the single driver by another name. None of this bears on richer effect systems [17]; it says these formulations add no rejection the single-driver rule lacks.

Finally, the dual form, which shows that the boundary between semantic behavior and physical realization is not a matter of taste. Once zero-input relationships have the canonical interface type $() \rightarrow A$ (normalized above the kernel to A , with the unit eliminated before any term is typed — the kernel has no unit type, and `delay` inside a zero-input declaration is why: a unit binder would forbid memory there, `delay_not_under_binder`), the form $A \rightarrow ()$ suggests itself as a consumer. It cannot name one: in a pure total language every function into the one-point type is the same function (`unit_codomain_collapse`, by function extensionality), so two “consumers” are indistinguishable (`consumers_indistinguishable`), and the evaluation relation has no effect component (`eval_`

independent_of_drives). Naming a receiver needs an output semantics, and the calculus already has exactly one. Semantic behavior and physical realization are related by the drive edge and are not identical.

9 REUSE OF RELATIONAL STRUCTURE

A second lamp should reuse the first’s behavior without copying it. What is reused is not a code module but a *structured set of relationships* — the lamp’s concepts, its declared relationships, their clocks and their outputs — with some relationships left open as ports. This section shows that everything a component needs is derivable from what §4 already provides, realization plus renaming: instantiation gives the relationships fresh identities, binding connects an instance into a larger design by ordinary realization steps, and flattening yields an ordinary design accepted by the unchanged judgments, so that composition adds no semantic machinery.

9.1 Equivariance

A renaming r bundles four maps — on declaration, concept, clock and output identities. Renaming acts on types (through the concepts C they mention), on terms, on interfaces, on declarations and pointwise on environments; Δ . **RenamedBy** r Δ' says Δ' stores the renamed declaration of Δ at the renamed identity.

PROPOSITION 18 (Equivariance; `HasType.rename`, `Satisfies.rename`, `Clocked.rename`). *If $\Theta; \Delta; \Gamma \vdash_G e : \tau$ and Θ', Δ', G' are the images of Θ, Δ, G under r (agreement on the image, with no injectivity required), then $\Theta'; \Delta'; \Gamma' \vdash_{G'} e^r : \tau^r$; likewise for satisfaction, and for the domain judgment under a clock environment that agrees on the declared identities.*

Proof. By induction on the derivation. Renaming acts homomorphically on types, terms, contexts and environments; each rule of Figure 2 is closed under it provided the environments agree on the image — T-Ref because Δ' holds the renamed declaration at the renamed name, T-Rep and T-Mk because Θ' binds the renamed concept to the renamed representation and the grant is transported, and the remaining rules syntactically. No injectivity is used: two names sent to one are consistent with every rule. Satisfaction and the clock judgment follow by the same induction, with equivariance of evidence as the hypothesis for the commitment half. $\square$

Evidence must be equivariant as well (`Evidence.Equivariant`), an abstract condition beside monotonicity. Nothing else is new in the composition theory; the rest is definitions over Proposition 18 and §4.

The running example, step 8 of Table 1. The three declarations become a component with `tilt` as its required port and `light` as its provided port. A second lamp is an instance: every name the template owns is renamed freshly, so its `dimByTilt` and `light` are new constants of the same interfaces, and binding its `tilt` port to a second sensor realizes that port with a reference — a `realize` step of §4, and nothing else.

9.2 Components, instances and flattening

A **port** is a template declaration by local identity with the public part of its interface and its parameter clock. A **behavior interface** has required ports (unrealized declarations a composer binds), provided ports, elaboration-time parameters (unrealized data-typed declarations bound to closed constants at instantiation) and clock parameters. A **component** is an interface, a template design over local identities below a width W , and a partition of its concepts and outputs into private (freshened per instance) and shared. The judgment **Realizes** $ev \mathcal{C}$ is a predicate over the existing judgments: the template is a well-formed design (`Design.WF`: `GlobalWF`, Θ . `WF`, well clocked, causal, `DriveWF`, `SingleDriver`), every required port is an unrealized declaration of the stated interface, every provided port is declared with it, parameters are unrealized, data-typed and clock-free.

Instance k of a component maps local identity n to fresh W $k \ n = W \cdot (k + 1) + n$, with `decode` its inverse; distinct instances never share an identity (`inst_decl_disjoint`). The encoding is a device — any injective allocator would do. A **binding** realizes a destination port of one instance from a source — a port of another instance or a closed constant — with an optional transport: none for a direct reference in the same or an agnostic domain, some *init* for `sync` from the source’s domain. A **system** is a width, a list of instances, a list of bindings, the shared concept environment and the external outputs. **Flattening** is the union of the renamed instances followed by

the bindings applied as §4 realization steps: the destination port is realized as src or $\text{sync}_\kappa \text{ init } (src)$. The result is a design, consumed by every existing judgment unchanged.

THEOREM 19 (Composition adds no machinery; `binding_satisfies`, `flatten_WF`, `flatten_causal`, `flatten_wellClocked`, `flatten_singleDriver`, `open_port_stays_open`). *Under `ComposeWF` — every instance realizes its interface; every binding is well formed (types agree; a direct binding’s source is in the destination’s domain or agnostic; a transported binding’s source has a domain); external outputs are driven by at most one instance — and with evidence that is monotone, equivariant and port-sound (a discharged commitment survives when a port copy is realized by a reference to a declaration of the same interface), the flattening is globally well formed, well clocked, single-driver, causal when the inter-instance graph is acyclic, and its open ports remain open.*

Proof. Flattening first takes the disjoint union of the instances, each template renamed by a fresh injective renaming, and then realizes every bound port with a binding body — a constant name, a transport of one, or a closed constant. Establish an invariant along the sequence of bindings: the current environment refines the union, is globally well formed, and holds every port already bound at its binding body. The base case is the union, globally well formed by Proposition 18 applied to each template. For a step, the binding body satisfies the destination port’s interface in any environment refining the union (`binding_satisfies`): a port binding reads a provided port whose interface refines the required one, with the transport’s initial value typed under the empty grant; a constant binding is closed and typed at the port’s type; the commitments transfer by port-soundness of the evidence. Realizing an unrealized port with a satisfying body is a `realize` step, so Theorem 4 preserves global well-formedness and the invariant. Causality of the result: order instances by the acyclic inter-instance graph and, within an instance, by the template’s own rank; a direct binding creates an instantaneous edge only from a destination to a source in a lower instance, and a transport creates none. Clocks: bindings connect ports of equal domain or through a transport whose source domain is the provided port’s. Single driver: each instance’s drive edges are renamed injectively and external sinks have at most one driving instance. An unbound port keeps no definiens, since only bindings realize ports. $\square$

THEOREM 20 (Modular semantics, restricted; `eval_flat_to_inst`, `eval_inst_to_flat`, `modular_iff_flat`). *For wiring designs with closure-free inputs and direct bindings, in one domain, the value of a declaration in an instance evaluated alone with a consistent modular input equals its value in the flattened system.*

Proof. In the modular semantics an instance evaluates its own declarations with an input that supplies, for each of its bound ports, the value the binding delivers; *consistent* means that input equals the flattened value of the port’s source. Forward direction, by induction on the flattened derivation over wiring terms (constants, applications of primitives, no lambdas): a reference the instance owns evaluates identically, since its definiens is the same renamed body; a reference to a bound port is, in the flattened design, a reference to the source, and its value is by consistency the modular input for that port. The backward direction is the same induction from the modular side, using totality of the flattened semantics to obtain the source’s value and consistency to identify it. Closure-freeness of inputs keeps every value first-order, so no environment of a closure can differ between the two sides; direct bindings keep every reference within one tick. The equivalence for a declaration δ owned by instance k is both directions at $e = \delta$. $\square$

The restriction is exact and recorded: transported bindings under MEV need a domain-indexed input for the transported port, and higher-order bodies are not covered — the same obstacle in both directions. Substitutability follows the usual contravariance: B may replace A when every port A provides, B provides at the same type and clock, and every port B requires, A required; replacing an instance by a refining component preserves `ComposeWF` (`substitute_composeWF`).

The counterexamples that fixed the design (`Experiments/BehaviorAlternatives.lean`): a name-based identity collides on double instantiation; a shared clock captured inside a template cannot be re-bound; a binding across domains without transport is rejected by `Clocked`; two instances driving one external output violate `SingleDriver`.

9.3 Groups are the identity, and extraction is a system

A *group* — a designer’s selection of several declarations — is authoring metadata: a group identity and a member list beside the design. Every group operation acts on the list and leaves the design untouched, so every kernel

judgment of the design is the *same proposition* before and after, each proved by reflexivity (`group_is_identity_on_design`). A group’s boundary is a projection over a finite enumeration: the non-members some member depends on (`crossIn`), the members some non-member depends on (`crossOut`); the aggregate socket a collapsed group shows is these lists, none of which is a declaration, and $a \in \text{crossIn}$ says *some* member depends on a and nothing about the others (`socket_no_fanout`). Two encodings of a socket as a declaration were refuted: as a declaration every member reads, a member acquires an instantaneous dependency it never had; as a tuple-returning declaration, the consumer of one member comes to depend on the inputs of all of them. The kernel has no tuples for boundaries, and this is a reason not to add them for that purpose.

Packaging a group as a component — the one semantic step in the hierarchy `declaration` $\rightarrow$ `group` $\rightarrow$ `component` $\rightarrow$ `system` — builds two *restrictions* of the design, the component with an unrealized copy of each crossing-in declaration as a required port and the residual with an unrealized copy of each crossing-out member, and forms a two-instance system with one direct binding per crossing. No realization is translated or copied across the boundary. Both templates realize their inferred interfaces (`restrict_realizes`, needing evidence that depends only on the interfaces of the referenced declarations, `Evidence.InterfaceLocal`); the system is a well-formed composition and its flattening a well-formed design (`system_composeWF`, `flat_WF`); causality needed its own argument, since a group with both inputs and outputs is never inter-instance-acyclic, yet the flattened instantaneous graph is the original with every crossing edge subdivided through a port copy, and doubling the original rank witnesses it (`flat_causal`); private members are unobservable from the residual (`private_unobservable`); and for wiring designs an original declaration and its home copy evaluate to the same value at every tick (`orig_iff_flat`).

10 MECHANIZATION

The development is 68 Lean 4 modules (Lean 4.33.1, no dependencies beyond core): 11 in *Core* (§3–6, §8), 12 in *Behavior* (§9), 19 in *Surface* (the definitional library, polymorphism, the window and the boundary constructions above the kernel), 2 in *Validation* (outside this paper), and 24 experiment modules holding alternatives, counterexamples and executed examples; about 26 500 lines and 1 545 theorem declarations. It builds with no sorry. The axioms are propositional extensionality and quotient soundness, the latter only through function extensionality and the choice-free rational quotient used by the unit laws; classical choice is absent, and the whole development was re-audited for it at every phase.

Three proof-engineering choices carried the metatheory. The relations `Ev` and `MEv` are ordinary inductive relations with no mutual recursion, because the recursor’s rule unrolls through the environment (§6.1); every induction on evaluation extends by one case when a construct is added, and the transport primitive and the recursor entered this way with every earlier theorem re-established without a change of statement. The logical relation is parameterized by an application relation so that the single- and multi-domain semantics share it, and is independent of that parameter at data types (`Red_data`), which is the fact that lets a value cross a tick. Every rejected alternative is a theorem whose content is a rejection, stated on a concrete design and discharged by `decide` or by running the interpreters `evalF/mevalF` — proved sound for the relations — inside the checker; there is no test suite beside the proofs. Several results are recorded as trivial by definition and reported as such. Extraction is not part of the development; the production toolchain implements the calculus in Rust and is tested differentially against the interpreter’s traces, a tested claim and not a theorem.

11 RELATED WORK

Modules and signatures. ML-style module systems separate an interface from its implementation, and a signature may be written, checked and depended upon before a structure matches it [2], [3]. The calculus does not claim that such systems cannot express an unrealized relationship. The difference is one of organization: here an unrealized declaration is an ordinary inhabitant of the *design environment* rather than a separate compilation unit; its clients are typed against it in the same environment and, by Theorem 3, remain typed when it is realized; and the same declaration is simultaneously the carrier of a nominal semantic signature (§5), a clock assignment (§6) and a drive edge (§8). The commitment list is a growable part of the interface whose growth is a first-class step on a declared-but-unrealized name, with a stability condition imposed on the layer that discharges it.

Refinement types, contracts and specification. Refinement type systems, contracts and specification languages already support progressively stronger constraints on a definition, and we do not claim to have invented refinement. BDL’s commitments are atomic labels, deliberately weaker than a refinement predicate; what is specific is where they attach — to a persistent relationship declaration whose realization may be absent — and what is proved about them: that a client’s discharged commitment survives later realization and strengthening under a monotonicity condition on evidence that is shown necessary (Theorems 4–5). The condition is the Kripke-style stability familiar from logical-relations proofs [11], [12], imposed here on a validation layer rather than on a store.

Synchronous and reactive languages. The temporal interpretation of §6 is that of Lustre [8] and Esterel [18]: a global logical tick, definitions as streams, memory as pre with an explicit initial value, causality as acyclicity of instantaneous dependencies; Vélus [19] verifies a compiler for this model, Zélus [20] and mode extensions [21] extend it. Functional reactive programming [22], [23], [24] makes signals first-class values, and typed FRP [25], [26], [27] controls memory through modal types. In all of these the central authored unit is a computation — a node, a stream definition, a signal function — and clocks are inferred from how it samples [9], [13], [28]. In λ_{BDL} the authored unit is a declared relationship, and the temporal machinery exists to interpret it in an authored domain: there is no signal type because a declaration already is a stream; the domain is a nominal identity chosen before any rate is known; and the one transport primitive with its strictly-before rule was chosen because the alternative exposes a scheduler the designer never authored (Theorem 15). The window buffer of §7.3 does the work that sub-sampling operators do in Lucid Synchronic, as a derived construction.

Nominal and abstract types; units of measure. Nominal type identity is the ordinary mechanism of a nominal type system [29], and the construction grant is the private constructor of an abstract type [4], [5]. Dimension types are Kennedy’s [6], [7], monomorphic in the kernel and instantiated by matching in surface families (§7.2). None of these is claimed as novel. Their role here is to keep a relationship between meanings distinct from a relationship between representations, and to keep representation arithmetic coherent — supporting mechanisms of the relationship-first model, with the erasure and provenance results (Proposition 8, Theorem 12) and the refuted alternatives (§5.1–5.2) as the evidence that they do that job.

Typed holes and live programming. Hazelnut [30], [31] gives a semantics to programs with holes and to the edit actions that fill them. An unrealized declaration is not a hole position in a term; it is a declaration whose realization is absent, referred to by identity and typed by its interface, and the refinement order plays the role of the edit-action calculus restricted to the operations under which clients are stable. The two are complementary: a hole is where a term is incomplete, an unrealized declaration is where a design is deliberately open.

Design and modeling languages. Block-diagram environments, model-based design and systems-modeling languages [32] also let a designer connect named quantities before every block is defined, and we do not claim that they lack relationships. What λ_{BDL} adds is a small mechanized calculus for the progressive-relationship model with explicit metatheory: what a client may depend on, what a realization may construct, when a value belongs to the design, and how reuse is derived — each as a theorem, and each rejected alternative as a counterexample.

Effects and expressiveness. The single-driver discipline is not an effect system [17]; §8 records that effect rows and action values, in the toy forms tried, add no rejection the drive edge lacks. The negative results of §7 are statements about whether a construct is definable from the kernel by a local translation, in the spirit of Felleisen’s expressiveness [33], as mechanized theorems about specific candidates.

12 DISCUSSION AND LIMITS

What “first-class” means here. A relationship is first-class as a *design object*: it has a stable identity, it may be declared, depended upon, refined, clocked, driven and instantiated, and every judgment of the calculus is stated over it. It is not a first-class *value*. Terms refer to declarations by identity and never pass a declaration as an argument or return one; there is no type of relationships, no higher-order manipulation of declarations, and the paper claims none. What is higher-order in the calculus is ordinary: functions over data, and the one recursor.

Design versus program. A program describes one computation. A design in λ_{BDL} may contain decisions at different levels of commitment — a relationship with a signature only, one with commitments, one with a realization, one with a clock and an output — and the calculus preserves that partially committed structure rather

than requiring it to be resolved before anything is checked. This is not a claim that conventional programs are fully specified, nor that their signatures lack meaning; it is that the *progression* from less to more committed is explicit here and has a metatheory.

One direction of commitment. The refinement order formalizes narrowing: more commitments, a realization, stronger verified commitments. It does not formalize the family of behaviors a design leaves open, and the calculus has no denotation of a set of possible products. Edits — changing a signature, dropping a commitment, replacing a realization — are outside the order, and the calculus promises nothing about them (Table 2).

The paper’s formal claims are further bounded by the following, each recorded in the development.

- *Causality is conservative for lambda-guarded cycles.* $A := \lambda x. A \times$ is rejected by Causal although $\text{declRef } A$ evaluates to a closure; Proposition 10 covers strict cycles only.
- *Modular semantics is proved for a fragment.* Theorem 20 holds for single-domain wiring designs with direct or constant bindings; transported bindings under MEV and higher-order realizations are open.
- *Evidence is abstract.* The kernel imposes monotonicity, equivariance and port-soundness on the validation layer’s evidence and proves nothing about a concrete discharge mechanism.
- *No sums.* Enumerations with payloads are encoded; a kernel sum would be one eliminator term former, and its absence is a decision to stop where the executed cases stopped.
- *Magnitudes are naturals.* The executable kernel’s quantities are natural numbers; the unit laws are proved over an abstract scalar domain and instantiated symbolically, and floating-point implementations are held to tolerated versions above the kernel.
- *No minimality theorem.* “Minimal” means minimal among the formalized candidates.
- *Nothing about designers.* The calculus was shaped by a design workflow’s requirements; whether it serves designers is an empirical question no theorem addresses, and no claim about cognitive load, productivity or ease is made.

13 CONCLUSION

The calculus is not interesting because it makes every implementation detail first-class. It is interesting because a typed semantic relationship can be declared, connected to concepts, depended upon by other parts of a product, placed in time and bound to a physical output while its computation is still undecided — and can then acquire that computation without disturbing anything built on it. The intermediate state is a design state, not a broken program state, and the calculus gives it a semantics in which it is typable, referenceable, composable, refinable, stable for clients and eventually realizable.

The results are the constraints on that one object, each with its theorem. Refinement preserves earlier reasoning: a client typed against a relationship’s promise survives its realization unconditionally, and a client’s discharged commitment survives it whenever evidence is positive in the environment, a condition that cannot be dropped (Theorems 3–5). Grants preserve semantic integrity: a realization may construct exactly the concept its signature announces, representation is not meaning, and no tag is manufactured by evaluation, memory or transport (Theorems 7, 12). Clocks preserve temporal meaning: a relationship’s value belongs to an authored domain, evaluation is deterministic and total on causal designs, and no scheduler order is observable (Theorems 9, 11, 14, 15). Outputs make physical effect explicit and single (Theorem 17). Renaming preserves relational structure under reuse, and composition adds no machinery (Theorems 19, 20). The mechanized counterexamples — a scheduler made visible, an evidence relation destroyed by a valid realization, a hidden crossing between concepts, three physical outputs from one design, a lossy summary for every bounded buffer — record why each constraint takes the form it does. That these constraints are all statements about a relationship, and that they have been proved to compose, is the conceptual unity of the calculus.

APPENDIX A — THEOREM INDEX

Every name is a Lean declaration in KCN-judu/BDL_FV; the file is given per group, and names are unqualified where the namespace is BDL.

paper	Lean name	file
Prop 1	infer_sound, infer_complete, HasType.unique	Core/Typing
§3.4	HasType.mono_env, HasType.mono_concept, HasType.mono_grant, HasType.weaken_append	Core/Typing
§4.1	InterfaceRefines_iff_semantic, naive_breaks_wellformedness	Core/Satisfaction, Experiments/DeclCounterexamples
Prop 2	DeclRefinesStar_iff, EnvRefines_update	Core/Satisfaction, Core/Decl
Thm 3	local_refinement_preserves_global_typing	Core/Env
Thm 4	local_refinement_preserves_global_wf, local_lifecycle_preserves_global_wf	Core/Env
Thm 5	badEv_not_mono, probe6_breaks	Experiments/DeclCounterexamples
Prop 6	Unfolds.det, Unfolds.exists_of_acyclic, Unfolds.not_of_cyclic, Unfolds.refFree_of_fullyRealized	Core/Dependency
Thm 7	HasType.constructs_granted, hidden_crossing_rejected_under_grant	Core/Typing, Experiments/RepresentationBindingAlternatives
Prop 8	HasType.erase, erase_not_injective, baseline_is_erased_modelA	Experiments/SemanticTypeAlternatives
§5.3	dimension_mismatch_rejected, counterexampleB_baseline_accepts_length_plus_time, same_dimension_does_not_imply_same_semantic_identity	Experiments/DimensionAlternatives
Thm 9	Ev.det, evalF_sound	Core/Reactive
Prop 10	Ev.not_of_strictCyclic, Causal_iff_acyclic_of_delayFree	Core/Reactive, Core/Dependency
Thm 11	fundamental, reactive_total, Ev.red, Red_data, Red_prim	Core/Reactive
§6.4	arrow_not_delayable, delay_not_under_binder, sync_not_under_binder, first_tick_undefined_without_init	Experiments/PolyAlternatives, Surface/UnitDomain, Experiments/ReactiveAlternatives
Thm 12	Ev.tag_provenance, temporal_state_preserves_semantic_identity	Core/Reactive
§6.5	unfolds_preserves_eval, Ev.pure, Ev.env_irrelevant, Ev.noClo	Core/Reactive
Prop 13	delay_is_sync_own, clocked_delay_iff_sync_own, single_domain_embedding	Core/Clock
Thm 14	MEv.det, mfundamental, multi_domain_total, mevalF_sound, MEv.tag_provenance	Core/Clock
Thm 15	scheduling_order_observable, equal_rate_not_same_domain, clocked_type_forces_polymorphism	Experiments/ClockAlternatives
Thm 16	buffer_window_correspondence, buffer_from_log_and_cursor, bounded_summary_not_lossless, lossless_iff_injective	Surface/Buffer, Core/Clock, Experiments/BufferAlternatives
§7.1	fold_total, mfold_total, fold_spec	Core/Reactive, Core/Clock, Surface/Stdlib
§7.1	churchfst_rank, church_pair_prenex_one_projection, Value.beq_iff, Cap.eq_iff_data, lt_only_on_quantities	Experiments/PolyAlternatives, Surface/Generic, Surface/Poly, Experiments/EquationExamples
§7.2	matchTy_sound, matchTy_complete, Scheme.instantiate_sound, generic_preserves_identity, lib_expansion	Surface/Poly, Surface/Generic, Surface/Stdlib
Thm 17	single_driver_output_deterministic, multiple_direct_drivers_rejected, hidden_arbitration_observable, first_output_binding_is_monotone	Core/Output, Experiments/OutputAlternatives
§8	driver_is_unit_domain, unit_codomain_collapse, consumers_indistinguishable, eval_independent_of_drives	Surface/UnitDomain
Prop 18	HasType.rename, Satisfies.rename, Clocked.rename	Behavior/Rename
Thm 19	inst_decl_disjoint, binding_satisfies, flatten_WF, flatten_causal, flatten_wellClocked, flatten_singleDriver, open_port_stays_open	Behavior/Instantiate, Behavior/Preservation
Thm 20	eval_flat_to_inst, eval_inst_to_flat, modular_iff_flat, substitute_composeWF	Behavior/Semantics, Behavior/Substitution
§9.3	group_is_identity_on_design, socket_no_fanout, restrict_realizes, system_composeWF, flat_WF, flat_causal, private_unobservable, orig_iff_flat	Behavior/Group, Behavior/Boundary, Behavior/Extract, Behavior/ExtractPreservation

REFERENCES

- [1] L. de Moura and S. Ullrich, “The Lean 4 Theorem Prover and Programming Language,” in *Automated Deduction – CADE 28*, in Lecture Notes in Computer Science, vol. 12699. Springer, 2021, pp. 625–635. doi: [10.1007/978-3-030-79876-5_37](https://doi.org/10.1007/978-3-030-79876-5_37).
- [2] R. Harper and M. Lillibridge, “A Type-Theoretic Approach to Higher-Order Modules with Sharing,” in *Proceedings of the 21st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ACM, 1994, pp. 123–137. doi: [10.1145/174675.176927](https://doi.org/10.1145/174675.176927).

- [3] X. Leroy, “Manifest Types, Modules, and Separate Compilation,” in *Proceedings of the 21st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ACM, 1994, pp. 109–122. doi: [10.1145/174675.176926](https://doi.org/10.1145/174675.176926).
- [4] J. C. Mitchell and G. D. Plotkin, “Abstract Types Have Existential Type,” *ACM Transactions on Programming Languages and Systems*, vol. 10, no. 3, pp. 470–502, 1988, doi: [10.1145/44501.45065](https://doi.org/10.1145/44501.45065).
- [5] J. C. Reynolds, “Types, Abstraction and Parametric Polymorphism,” in *Information Processing 83, Proceedings of the IFIP 9th World Computer Congress*, North-Holland, 1983, pp. 513–523.
- [6] A. Kennedy, “Relational Parametricity and Units of Measure,” in *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ACM, 1997, pp. 442–455. doi: [10.1145/263699.263761](https://doi.org/10.1145/263699.263761).
- [7] A. Kennedy, “Types for Units-of-Measure: Theory and Practice,” in *Central European Functional Programming School*, in *Lecture Notes in Computer Science*, vol. 6299. Springer, 2010, pp. 268–305. doi: [10.1007/978-3-642-17685-2_8](https://doi.org/10.1007/978-3-642-17685-2_8).
- [8] N. Halbwachs, P. Caspi, P. Raymond, and D. Pilaud, “The Synchronous Data Flow Programming Language LUSTRE,” *Proceedings of the IEEE*, vol. 79, no. 9, pp. 1305–1320, 1991, doi: [10.1109/5.97300](https://doi.org/10.1109/5.97300).
- [9] J.-L. Colaço and M. Pouzet, “Clocks as First Class Abstract Types,” in *Proceedings of the Third ACM International Conference on Embedded Software*, ACM, 2003.
- [10] G. Kahn, “The Semantics of a Simple Language for Parallel Programming,” *Information Processing 74*, pp. 471–475, 1974.
- [11] A. W. Appel and D. McAllester, “An Indexed Model of Recursive Types for Foundational Proof-Carrying Code,” *ACM Transactions on Programming Languages and Systems*, vol. 23, no. 5, pp. 657–683, 2001, doi: [10.1145/504709.504712](https://doi.org/10.1145/504709.504712).
- [12] A. Ahmed, “Step-Indexed Syntactic Logical Relations for Recursive and Quantified Types,” in *Programming Languages and Systems, 15th European Symposium on Programming*, in *Lecture Notes in Computer Science*, vol. 3924. Springer, 2006, pp. 69–83. doi: [10.1007/11693024_6](https://doi.org/10.1007/11693024_6).
- [13] D. Biernacki, J.-L. Colaço, G. Hamon, and M. Pouzet, “Clock-Directed Modular Code Generation for Synchronous Data-Flow Languages,” in *Proceedings of the 2008 ACM SIGPLAN-SIGBED Conference on Languages, Compilers, and Tools for Embedded Systems*, ACM, 2008, pp. 121–130. doi: [10.1145/1375657.1375674](https://doi.org/10.1145/1375657.1375674).
- [14] L. Damas and R. Milner, “Principal Type-Schemes for Functional Programs,” in *Proceedings of the 9th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ACM, 1982, pp. 207–212. doi: [10.1145/582153.582176](https://doi.org/10.1145/582153.582176).
- [15] B. C. Pierce and D. N. Turner, “Local Type Inference,” *ACM Transactions on Programming Languages and Systems*, vol. 22, no. 1, pp. 1–44, 2000, doi: [10.1145/345099.345100](https://doi.org/10.1145/345099.345100).
- [16] P. Wadler, “Theorems for Free!,” in *Proceedings of the Fourth International Conference on Functional Programming Languages and Computer Architecture*, ACM, 1989, pp. 347–359. doi: [10.1145/99370.99404](https://doi.org/10.1145/99370.99404).
- [17] G. D. Plotkin and M. Pretnar, “Handling Algebraic Effects,” *Logical Methods in Computer Science*, vol. 9, no. 4, 2013, doi: [10.2168/LMCS-9\(4:23\)2013](https://doi.org/10.2168/LMCS-9(4:23)2013).
- [18] G. Berry and G. Gonthier, “The Esterel Synchronous Programming Language: Design, Semantics, Implementation,” *Science of Computer Programming*, vol. 19, no. 2, pp. 87–152, 1992, doi: [10.1016/0167-6423\(92\)90005-V](https://doi.org/10.1016/0167-6423(92)90005-V).
- [19] T. Bourke, L. Brun, P.-É. Dagand, X. Leroy, M. Pouzet, and L. Rieg, “A Formally Verified Compiler for Lustre,” in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ACM, 2017, pp. 586–601. doi: [10.1145/3062341.3062358](https://doi.org/10.1145/3062341.3062358).
- [20] T. Bourke and M. Pouzet, “Zélus: A Synchronous Language with ODEs,” in *Proceedings of the 16th International Conference on Hybrid Systems: Computation and Control*, ACM, 2013, pp. 113–118. doi: [10.1145/2461328.2461348](https://doi.org/10.1145/2461328.2461348).
- [21] J.-L. Colaço, B. Pagano, and M. Pouzet, “A Conservative Extension of Synchronous Data-flow with State Machines,” in *Proceedings of the 5th ACM International Conference on Embedded Software*, ACM, 2005. doi: [10.1145/1086228.1086261](https://doi.org/10.1145/1086228.1086261).
- [22] C. Elliott and P. Hudak, “Functional Reactive Animation,” in *Proceedings of the Second ACM SIGPLAN International Conference on Functional Programming*, ACM, 1997, pp. 263–273. doi: [10.1145/258948.258973](https://doi.org/10.1145/258948.258973).

- [23] H. Nilsson, A. Courtney, and J. Peterson, “Functional Reactive Programming, Continued,” in *Proceedings of the 2002 ACM SIGPLAN Workshop on Haskell*, ACM, 2002, pp. 51–64. doi: [10.1145/581690.581695](https://doi.org/10.1145/581690.581695).
- [24] G. H. Cooper and S. Krishnamurthi, “Embedding Dynamic Dataflow in a Call-by-Value Language,” in *Programming Languages and Systems: 15th European Symposium on Programming*, Springer, 2006, pp. 294–308.
- [25] N. R. Krishnaswami, “Higher-Order Functional Reactive Programming without Spacetime Leaks,” in *Proceedings of the 18th ACM SIGPLAN International Conference on Functional Programming*, ACM, 2013, pp. 221–232. doi: [10.1145/2500365.2500588](https://doi.org/10.1145/2500365.2500588).
- [26] A. Jeffrey, “LTL Types FRP: Linear-Time Temporal Logic Propositions as Types, Proofs as Functional Reactive Programs,” in *Proceedings of the Sixth Workshop on Programming Languages meets Program Verification*, ACM, 2012, pp. 49–60. doi: [10.1145/2103776.2103783](https://doi.org/10.1145/2103776.2103783).
- [27] A. Cave, F. Ferreira, P. Panangaden, and B. Pientka, “Fair Reactive Programming,” in *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ACM, 2014, pp. 361–372. doi: [10.1145/2535838.2535881](https://doi.org/10.1145/2535838.2535881).
- [28] P. Caspi and M. Pouzet, “Synchronous Kahn Networks,” in *Proceedings of the First ACM SIGPLAN International Conference on Functional Programming*, ACM, 1996, pp. 226–238. doi: [10.1145/232627.232651](https://doi.org/10.1145/232627.232651).
- [29] B. C. Pierce, *Types and Programming Languages*. MIT Press, 2002.
- [30] C. Omar, I. Voysey, M. Hilton, J. Aldrich, and M. A. Hammer, “Hazelnut: A Bidirectionally Typed Structure Editor Calculus,” in *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, ACM, 2017, pp. 86–99. doi: [10.1145/3009837.3009900](https://doi.org/10.1145/3009837.3009900).
- [31] C. Omar, I. Voysey, R. Chugh, and M. A. Hammer, “Live Functional Programming with Typed Holes,” *Proceedings of the ACM on Programming Languages*, vol. 3, no. POPL, p. 14:1–14:32, 2019, doi: [10.1145/3290327](https://doi.org/10.1145/3290327).
- [32] D. Harel, “Statecharts: A Visual Formalism for Complex Systems,” *Science of Computer Programming*, vol. 8, no. 3, pp. 231–274, 1987, doi: [10.1016/0167-6423\(87\)90035-9](https://doi.org/10.1016/0167-6423(87)90035-9).
- [33] M. Felleisen, “On the Expressive Power of Programming Languages,” in *Programming Languages and Systems, 3rd European Symposium on Programming*, in *Lecture Notes in Computer Science*, vol. 432. Springer, 1990, pp. 134–151. doi: [10.1007/3-540-52592-0_60](https://doi.org/10.1007/3-540-52592-0_60).